

ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ



Lê Văn Đức

PHƯƠNG PHÁP XÁC ĐỊNH DỊCH VỤ GÂY LỖI
TRONG HỆ THỐNG VI DỊCH VỤ

KHOÁ LUẬN TỐT NGHIỆP

Ngành: Khoa học máy tính

HÀ NỘI - 2026

ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ



LÊ VĂN ĐỨC

PHƯƠNG PHÁP XÁC ĐỊNH DỊCH VỤ GÂY LỖI
TRONG HỆ THỐNG VI DỊCH VỤ

KHOÁ LUẬN TỐT NGHIỆP

Ngành: Khoa học máy tính

Cán bộ hướng dẫn: PGS. TS. Võ Đình Hiếu

HÀ NỘI - 2026

**VIETNAM NATIONAL UNIVERSITY, HANOI
UNIVERSITY OF ENGINEERING AND TECHNOLOGY**



Le Van Duc

**A METHOD FOR IDENTIFYING FAULTY SERVICES IN
MICROSERVICES SYSTEMS**

BACHELOR'S THESIS

Major: Computer Science

Supervisor: Assoc. Prof. Dr. Vo Dinh Hieu

HANOI - 2026

Lời cảm ơn

Lời đầu tiên, tôi xin gửi lời cảm ơn chân thành và lòng biết ơn sâu sắc tới thầy Võ Đình Hiếu vì đã hướng dẫn tận tình và hỗ trợ tôi về kiến thức chuyên môn, những lời đánh giá, cũng như những định hướng đắt giá trong quá trình nghiên cứu và thực hiện khoá luận này.

Tôi cũng xin gửi lời cảm ơn đến các anh chị, các bạn trong nhóm dẫn hỗ trợ tôi rất nhiều kiến thức chuyên môn trong quá trình thực hiện khoá luận.

Cuối cùng, tôi xin gửi lời cảm ơn chân thành tới các thầy, cô hiện đang công tác và giảng dạy tại trường Đại học Công nghệ - Đại học Quốc gia Hà Nội đã tạo mọi điều kiện thuận lợi cho tôi học tập và nghiên cứu.

Lời cam đoan

Tôi là Lê Văn Đức, sinh viên lớp QH-2022-I/CQ-I-CS3 khoá K67 theo học ngành Khoa học máy tính tại trường Đại học Công nghệ - Đại học Quốc gia Hà Nội, Tôi xin cam đoan khoá luận "Phương pháp xác định dịch vụ gây lỗi trong hệ thống vi dịch vụ" là công trình nghiên cứu do bản thân tôi thực hiện. Các nội dung nghiên cứu, kết quả trong báo cáo là xác thực.

Các thông tin sử dụng trong báo cáo là có cơ sở và không có nội dung nào sao chép từ các tài liệu mà không rõ trích dẫn tham khảo. Trong quá trình thực hiện khoá luận, tôi có sử dụng trí tuệ nhân tạo như một công cụ hỗ trợ trong các bước thực hiện phân tích, đánh giá và xây dựng ý tưởng, tuy nhiên không có sự can thiệp quá sâu trong quá trình thực hiện cài đặt thuật toán. Tôi xin chịu trách nhiệm về lời cam đoan này.

Hà Nội, ngày tháng năm

Sinh viên

Lê Văn Đức

Tóm tắt

Trong các hệ thống vi dịch vụ ở thời điểm hiện tại, quá trình giám sát thường sinh ra số lượng lớn dữ liệu trace nhằm phục vụ quan sát trạng thái hệ thống và phân tích sự cố. Tuy nhiên, việc lưu trữ và xử lý toàn bộ trace gây ra chi phí đáng kể về tài nguyên tính toán và lưu trữ. Vì vậy, lấy mẫu trace là một hướng tiếp cận quan trọng để giảm tải dữ liệu quan sát. Vấn đề đặt ra là phương pháp lấy mẫu không chỉ cần giảm số lượng trace được giữ lại, mà còn phải bảo toàn các tín hiệu quan trọng phục vụ phân tích nguyên nhân gốc trong hệ thống vi dịch vụ.

Khóa luận này nghiên cứu bài toán lấy mẫu trace thích nghi trong bối cảnh phân tích nguyên nhân gốc. Phương pháp đề xuất xây dựng một pipeline hai pha gồm: lấy mẫu trace từ luồng dữ liệu ban đầu và đánh giá chất lượng phân tích nguyên nhân gốc trên tập trace sau lấy mẫu. Trong pha lấy mẫu, mỗi trace được biểu diễn thành các đặc trưng phục vụ chấm điểm, bao gồm thông tin đường đi dịch vụ, độ trễ, lỗi và các tín hiệu liên quan đến ngữ cảnh sự cố. Trên cơ sở đó, khóa luận đề xuất các chiến lược lấy mẫu thích nghi nhằm ưu tiên những trace có khả năng mang thông tin quan trọng cho RCA, đồng thời duy trì tính đa dạng của tập trace được giữ lại dưới các mức ngân sách lấy mẫu khác nhau. Trong pha đánh giá, tập trace sau lấy mẫu được đưa vào MicroRank để xếp hạng các dịch vụ nghi ngờ là nguyên nhân gốc, từ đó đo lường tác động của phương pháp lấy mẫu đến chất lượng RCA.

Các thí nghiệm được thực hiện trên bộ dữ liệu của hệ thống vi dịch vụ với nhiều mức ngân sách lấy mẫu. Kết quả cho thấy phương pháp đề xuất có khả năng giảm đáng kể số lượng trace cần xử lý trong khi vẫn bảo toàn được các tín hiệu quan trọng cho phân tích nguyên nhân gốc. So với các phương pháp lấy mẫu cơ bản, các biến thể lấy mẫu thích nghi trong khóa luận cho thấy hiệu quả tốt hơn trong việc duy trì chất lượng xếp hạng RCA, đặc biệt ở các mức ngân sách nhỏ. Ngoài ra, phân tích chi phí xử lý cũng cho thấy phương pháp có thể hoạt động với độ trễ thấp trên mỗi trace, phù hợp với mục tiêu giảm tải tài nguyên trong các hệ thống giám sát thực tế.

Từ khóa: Khả năng quan sát, hệ thống vi dịch vụ, lấy mẫu trace, phân tích nguyên nhân gốc, phương pháp lấy mẫu, tiết kiệm tài nguyên.

Abstract

Modern microservice systems generate a large volume of trace data for system monitoring, observability, and incident analysis. However, storing and processing all traces may introduce significant overhead in terms of computation, memory, and storage resources. Therefore, trace sampling is an important approach for reducing observability data while maintaining useful information for downstream analysis. In the context of root cause analysis, the sampling objective is not only to reduce the number of retained traces, but also to preserve important signals that help identify the faulty service accurately.

This thesis investigates adaptive trace sampling for root cause analysis in microservice systems. The proposed approach follows a two-phase pipeline, including trace sampling from the original trace stream and evaluating root cause analysis using the sampled traces. In the sampling phase, each trace is transformed into a set of features, including service invocation paths, latency, error information, and incident-related contextual signals. Based on these features, the thesis proposes adaptive sampling strategies that prioritize traces carrying important RCA-related information while maintaining diversity under different sampling budgets. In the evaluation phase, the sampled traces are used as input to MicroRank, which ranks suspicious services according to their likelihood of being the root cause.

Experiments are conducted on microservice trace datasets under multiple sampling budgets. The results show that the proposed method can significantly reduce the number of traces that need to be processed while preserving important signals for root cause analysis. Compared with basic sampling methods, the adaptive sampling variants proposed in this thesis achieve better RCA ranking quality, especially under low sampling budgets. In addition, the processing cost analysis shows that the proposed method maintains low per-trace latency, indicating its potential applicability to practical observability systems with limited resource budgets.

Keywords: Observability, microservice systems, trace sampling, root cause analysis, sampling method, resource efficiency.

Mục lục

Lời cảm ơn

Lời cam đoan i

Tóm tắt ii

Abstract iii

Mục lục iv

Danh sách hình vẽ viii

Danh sách bảng ix

Danh mục các từ viết tắt x

Chương 1 Đặt vấn đề 1

1.1 Giới thiệu về bài toán 1

1.2 Các nghiên cứu liên quan 3

1.3 Mục tiêu và đóng góp của khoá luận 6

1.4 Cấu trúc khoá luận 8

Chương 2 Cơ sở lý thuyết và nền tảng bài toán 9

2.1 Hệ thống vi dịch vụ và dữ liệu quan sát 9

2.1.1 Kiến trúc hệ thống vi dịch vụ 9

2.1.2 Dữ liệu quan sát trong hệ thống phân tán 10

2.1.3 Truy vết phân tán 10

2.2 Dữ liệu truy vết phân tán 11

2.2.1 Span và trace 11

2.2.2 Cấu trúc topo của trace 12

2.2.3 Các đặc trưng của trace 12

2.2.4	Đặc điểm và thách thức của dữ liệu trace	13
2.3	Phân tích nguyên nhân gốc rễ trong hệ thống phân tán	13
2.3.1	Khái niệm phân tích nguyên nhân gốc rễ	13
2.3.2	RCA dựa trên phổ lỗi	14
2.3.3	MicroRank	15
2.3.4	Các độ đo đánh giá chất lượng RCA	15
2.4	Bài toán lấy mẫu trace phục vụ RCA	16
2.4.1	Lấy mẫu trace dưới ràng buộc ngân sách	16
2.4.2	Lấy mẫu theo dòng	17
2.4.3	Mục tiêu lấy mẫu nhận biết RCA	17
2.4.4	Ảnh hưởng của sampling đến chất lượng RCA	18
2.4.5	Thách thức ở mức ngân sách thấp	18
2.4.6	Bài toán ổn định khi mở rộng ngân sách	19
2.5	Nền tảng kỹ thuật cho lựa chọn trace	19
2.5.1	Phát hiện bất thường trong dữ liệu hệ thống	19
2.5.2	Đo lường độ tương đồng và đa dạng trace	20
2.5.3	Xây dựng hàm đánh giá trace	21
2.5.4	Kỹ thuật tổ hợp (ensemble) và đồng thuận (consensus)	21

Chương 3 Phương pháp lấy mẫu trace thích nghi nhận biết RCA bằng đồng thuận đối sánh

22

3.1	Tổng quan phương pháp và kiến trúc hệ thống	23
3.1.1	Định hướng và cách tiếp cận	23
3.1.2	Pipeline hai pha	24
3.1.3	Biểu diễn trace đầu vào	24
3.1.4	Ràng buộc xử lý theo dòng	25
3.2	Phương pháp nền TASD: lấy mẫu trace thích nghi dựa trên chấm điểm và đa dạng	26
3.2.1	Mục tiêu thiết kế	26
3.2.2	Các đặc trưng trace và tín hiệu chấm điểm	27
3.2.3	Tín hiệu trạng thái hệ thống suy ra từ trace	28
3.2.4	Đa dạng hóa trace bằng độ tương đồng Jaccard	29
3.2.5	Hàm chấm điểm tổng hợp và quy tắc chọn trace	30
3.2.6	Hạn chế của TASD	31

3.3	Mở rộng chấm điểm đối sánh cho MicroRank	31
3.3.1	Vì sao MicroRank cần đối sánh	31
3.3.2	Mô hình bốn vùng RCA	32
3.3.3	Chấm điểm đối sánh RCA	33
3.3.4	Điều tiết tỷ lệ trace lỗi và trace bình thường	33
3.4	Phương pháp MESC: lấy mẫu đối sánh có dấu dựa trên metric gốc và đồng thuận tổ hợp	34
3.4.1	Động cơ đưa metric gốc vào bộ lấy mẫu	34
3.4.2	Ánh xạ trace với metric theo dịch vụ và thời gian	35
3.4.3	Đối sánh có dấu	36
3.4.4	Tổ hợp nhiều thành viên	36
3.4.5	Cơ chế đồng thuận	37
3.4.6	Quy trình chọn trace cuối cùng	38
3.4.7	So sánh TASD và MESC	39
3.5	Không gian thiết kế và các biến thể mô hình	40
3.5.1	Không gian thiết kế	41
3.5.2	Các biến thể và cấu hình thực nghiệm	42
3.6	Tích hợp với MicroRank cho phân tích nguyên nhân gốc	43
3.6.1	Đầu vào và đầu ra của MicroRank	43
3.6.2	Ảnh hưởng của việc lấy mẫu đến chất lượng RCA	43
Chương 4	Thực nghiệm và Đánh giá	45
4.1	Môi trường và dữ liệu thực nghiệm	45
4.1.1	Cấu hình môi trường và pipeline thực nghiệm	45
4.1.2	Bộ dữ liệu thực nghiệm	46
4.1.3	Biểu diễn trace và trích xuất kịch bản sự cố	48
4.2	Phương pháp và chỉ số đánh giá	48
4.3	Cài đặt các chiến lược lấy mẫu	49
4.3.1	Ràng buộc về ngân sách	49
4.3.2	Các phương pháp đối chứng	49
4.3.3	Cấu hình các phương pháp đề xuất	50
4.4	Kết quả thực nghiệm	50
4.4.1	Đánh giá kết quả của TASD	51
4.4.2	Đánh giá kết quả của MESC	54

4.4.3 Phân tích theo từng bộ dữ liệu	56
4.4.4 So sánh chi phí thời gian	57
4.4.5 Tổng hợp nhận xét theo câu hỏi nghiên cứu	59
Kết luận	62
Tài liệu tham khảo	65

Danh sách hình vẽ

1.1	Mô tả lấy mẫu đầu luồng (Head-based Sampling) [1].	3
1.2	Mô tả lấy mẫu cuối luồng (Tail-based Sampling) [1].	4
2.1	Mối quan hệ giữa span và trace, cấu trúc topology của trace [2]. . . .	11
2.2	Minh hoạ cấu trúc microrank [3].	15
3.1	Pipeline hai pha của phương pháp đề xuất.	22
3.2	Quy trình lấy mẫu TASD dựa trên tập trace gốc.	26
3.3	Đa dạng hóa trace bằng độ tương đồng Jaccard giữa các chuỗi dịch vụ. .	29
3.4	Quy trình lấy mẫu MESC kết hợp metric gốc.	38
3.5	Ảnh hưởng của tập trace sau lấy mẫu đến MicroRank.	44
4.1	So sánh khoảng Min–Max thời gian lấy mẫu trên mỗi trace giữa TraStrainer và MESC.	58
4.2	Tổng chi phí xử lý của MESC trên ba bộ dữ liệu và ba mức ngân sách. .	59

Danh sách bảng

2.1	Ma trận thống kê sự xuất hiện của một dịch vụ trong các trace bất thường và bình thường.	14
3.1	Các nhóm tín hiệu trong T ASD.	27
3.2	Mô hình bốn vùng RCA cho trace sampling.	32
3.3	So sánh T ASD và MESC.	39
3.4	Các trục chính trong không gian thiết kế của phương pháp lấy mẫu.	41
3.5	Các biến thể chính được đánh giá thực nghiệm trong khóa luận.	42
4.1	Tổng quan ba bộ dữ liệu thực nghiệm.	47
4.2	So sánh T ASD và MESC với các phương pháp lấy mẫu khác trên ba mức ngân sách.	51
4.3	Kết quả $A@1$, $A@3$ và MRR của T ASD và MESC trên từng bộ dữ liệu.	56
4.4	Chi phí thời gian của sampler trên ba bộ dữ liệu và ba mức ngân sách.	57

Danh mục các từ viết tắt

Từ viết tắt	Cụm từ đầy đủ	Ý nghĩa / Mô tả
RCA	Root Cause Analysis	Phân tích nguyên nhân gốc rễ
A@1	Accuracy@1	Độ chính xác top-1
A@3	Accuracy@3	Độ chính xác top-3
MRR	Mean Reciprocal Rank	Hạng nghịch đảo trung bình
CPU	Central Processing Unit	Bộ xử lý trung tâm
API	Application Programming Interface	Giao diện lập trình ứng dụng
MAD	Median Absolute Deviation	Độ lệch tuyệt đối trung vị
EMA	Exponential moving average	Trung bình động hàm mũ
TASD	Trace-Adaptive Scoring with Diversity	Lấy mẫu trace thích nghi dựa trên chấm điểm và đa dạng
MESC	Metric-native Ensemble Signed-Contrast Sampling	Lấy mẫu đối sánh có dấu dựa trên metric gốc và tổ hợp đồng thuận

Chương 1

Đặt vấn đề

Trong những năm gần đây, cùng với sự phát triển mạnh mẽ của công nghệ thông tin, và nhu cầu ứng dụng phần mềm trong nhiều lĩnh vực, kiến trúc vi dịch vụ (microservice) đã xuất hiện và dần trở thành một hướng tiếp cận phổ biến trong phát triển các hệ thống phần mềm quy mô lớn. Thay vì tổ chức toàn bộ ứng dụng dưới dạng một khối duy nhất (monolith), kiến trúc vi dịch vụ phân rã hệ thống thành nhiều dịch vụ nhỏ, mỗi dịch vụ đảm nhiệm một chức năng nghiệp vụ riêng và có thể được phát triển, triển khai, mở rộng một cách tương đối độc lập [4, 5]. Tuy nhiên sự phân rã trong việc phát triển và lựa chọn công nghệ cũng làm tăng độ phức tạp trong quá trình vận hành hệ thống, đặc biệt là việc giám sát và theo dõi trạng thái hệ thống theo thời gian thực. Các phương pháp hiện nay vẫn phải đối mặt với những bài toán về xử lý khối lượng lớn dữ liệu trạng thái của hệ thống một cách nhanh chóng và chính xác nhưng vẫn đảm bảo việc tối ưu và tiết kiệm tài nguyên ở một ngưỡng có thể chấp nhận [6].

1.1 Giới thiệu về bài toán

Trong môi trường vi dịch vụ, lỗi hoặc sự suy giảm hiệu năng tại một dịch vụ có thể lan truyền sang các dịch vụ phụ thuộc và biểu hiện thành lỗi ở những vị trí khác trong chuỗi xử lý. Khi chỉ quan sát kết quả cuối cùng, hệ thống có thể ghi nhận một yêu cầu thất bại hoặc độ trễ cao, trong khi nguyên nhân thực sự lại nằm ở một dịch vụ nằm sâu trong đường đi của yêu cầu (request). Vì vậy, bài toán phân tích nguyên nhân gốc rễ (Root Cause Analysis – RCA), đóng vai trò quan trọng trong vận hành hệ thống vi dịch vụ. Từ các dữ liệu quan sát được, RCA cần đưa ra danh sách các dịch vụ nghi vấn được xếp hạng theo khả năng là nguyên nhân gốc rễ của sự cố [3, 7].

Theo dõi phân tán (Distributed Tracing) là một kỹ thuật quan trọng để theo dõi đường đi của yêu cầu trong các hệ thống phân tán (Distributed System). Trong ngữ cảnh này, một **trace** có thể được hiểu là *vết truy vết của một yêu cầu*, ghi lại

toàn bộ hành trình của một yêu cầu. Trong một *trace*, mỗi *span* biểu diễn một đơn vị xử lý cụ thể, ví dụ như một lời gọi API, một truy vấn cơ sở dữ liệu hoặc một lời gọi giữa hai dịch vụ [6]. Bên cạnh dữ liệu *trace*, hệ thống quan sát cũng thường thu thập các **metric**, tức các số liệu đo lường trạng thái hệ thống theo thời gian, chẳng hạn như độ trễ, lưu lượng yêu cầu, tỷ lệ lỗi, mức sử dụng của bộ xử lý trung tâm (Central Processing Unit – CPU) hoặc bộ nhớ. Thông qua dữ liệu *trace* và *metric*, các hệ thống giám sát có thể thấy rõ luồng gọi giữa các dịch vụ, thời gian xử lý cũng như các dấu hiệu lỗi. Những thông tin này chính là nền tảng cốt lõi cho các phương pháp phát hiện bất thường và định vị nguyên nhân lỗi trong kiến trúc vi dịch vụ [3, 7, 8].

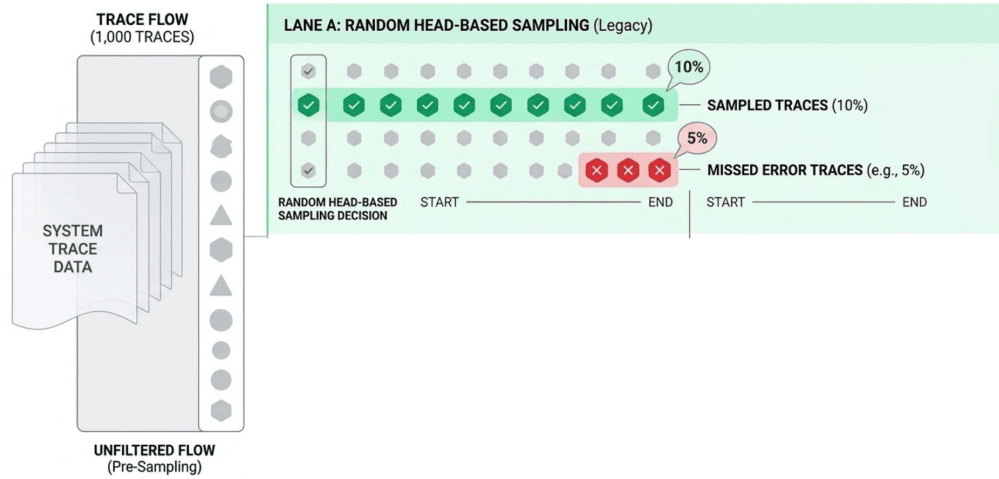
Tuy nhiên, việc sử dụng theo dõi phân tán trong thực tế cũng đặt ra thách thức lớn về chi phí. Trong các hệ thống vi dịch vụ, số lượng *trace* được sinh ra có thể rất lớn, trong đó nhiều *trace* mang hành vi tương tự nhau hoặc không chứa thông tin hữu ích cho quá trình chẩn đoán lỗi. Nếu lưu trữ và phân tích toàn bộ, hệ thống sẽ phải chịu gánh nặng đáng kể về băng thông, không gian lưu trữ và tài nguyên xử lý. Vì vậy, các hệ thống thường áp dụng cơ chế lấy mẫu *trace* nhằm giữ lại một phần dữ liệu theo một ngân sách nhất định [9, 10].

Đối với bài toán RCA, lấy mẫu *trace* không chỉ nhằm giảm số lượng dữ liệu được lưu trữ, mà còn phải đảm bảo rằng phần dữ liệu được giữ lại vẫn đủ giá trị cho quá trình phân tích nguyên nhân gốc. Một dịch vụ xuất hiện lỗi trong *trace* chưa chắc là nguyên nhân ban đầu, mà có thể chỉ là thành phần bị ảnh hưởng bởi lỗi từ dịch vụ khác. Ngược lại, dịch vụ gây lỗi có thể không trực tiếp sinh ra trạng thái lỗi rõ ràng, mà chỉ biểu hiện thông qua độ trễ hoặc ảnh hưởng gián tiếp lên các dịch vụ phụ thuộc. Vì vậy, RCA trong hệ thống vi dịch vụ không thể chỉ dựa vào việc đếm số lỗi tại từng dịch vụ, mà cần xem xét đồng thời cấu trúc phụ thuộc, sự khác biệt giữa *trace* bình thường và *trace* bất thường, cũng như mức độ ảnh hưởng của từng dịch vụ trong toàn bộ luồng xử lý [3, 11].

Từ các thách thức trên, có thể xem bài toán xác định nguyên nhân gốc trong hệ thống vi dịch vụ là một bài toán xếp hạng trong điều kiện dữ liệu lớn, nhiều tác nhân gây nhiễu, có quan hệ phụ thuộc phức tạp và bị giới hạn bởi ngân sách thu thập dữ liệu. Đầu vào của bài toán bao gồm tập *trace*, các *span* tương ứng, đồ thị phụ thuộc giữa các dịch vụ và dữ liệu trạng thái hệ thống thu thập theo thời gian thực. Đầu ra là danh sách các dịch vụ được xếp hạng theo khả năng là nguyên

nhân gốc của sự cố. Một phương pháp hiệu quả cần vừa tận dụng được thông tin cấu trúc của hệ thống, vừa khai thác được các dấu hiệu bất thường trong *trace*, *metric* đồng thời vẫn hoạt động tốt khi dữ liệu đầu vào chỉ là một phần được lấy mẫu từ toàn bộ dữ liệu tracing.

1.2 Các nghiên cứu liên quan

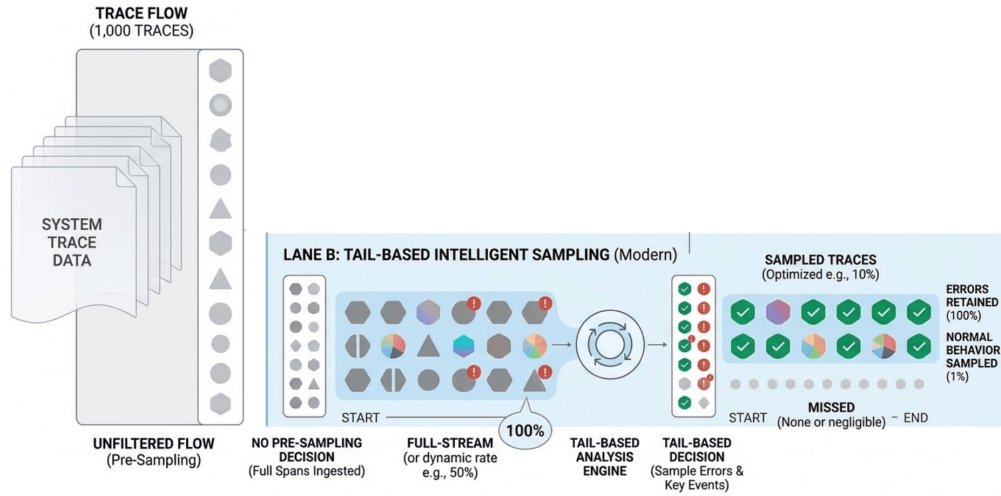


Hình 1.1: Mô tả lấy mẫu đầu luồng (Head-based Sampling) [1].

Bài toán giảm chi phí lưu trữ và xử lý dữ liệu trace trong hệ thống vi dịch vụ đã được nghiên cứu theo nhiều hướng khác nhau [12–14]. Một nhóm phương pháp phổ biến là **lấy mẫu ở đầu luồng xử lý** (head-based sampling). Các phương pháp như lấy mẫu ngẫu nhiên (random sampling) hoặc giới hạn tốc độ lấy mẫu (rate-limiting sampling) như minh hoạ trong Hình 1.1 thường đưa ra quyết định giữ hoặc loại bỏ trace ngay khi yêu cầu mới được tạo ở phía ứng dụng hoặc tác tử thu thập (agent), tức là trước khi toàn bộ trace được hoàn thành và trước khi bộ thu thập dữ liệu (collector) có thể quan sát đầy đủ các span của trace [9, 10].

Ưu điểm của hướng tiếp cận này là đơn giản, dễ triển khai và tạo ra rất ít chi phí phụ trong quá trình vận hành, do đã giảm thiểu số lượng ngay từ đầu. Tuy nhiên, do quyết định lấy mẫu được thực hiện trước khi toàn bộ trace hoàn thành, các phương pháp này không quan sát được những dấu hiệu quan trọng như lỗi, sự bất thường hoặc đường đi thực tế của yêu cầu trong toàn bộ hệ thống. Do đó, trong nhiều trường hợp, các trace có giá trị cao cho việc đánh giá sự cố có thể bị loại bỏ

từ sớm, trong khi nhiều trace bình thường và trùng lặp vẫn được giữ lại [13, 14].



Hình 1.2: Mô tả lấy mẫu cuối luồng (Tail-based Sampling) [1].

Để khắc phục hạn chế trên, một hướng tiếp cận khác là **lấy mẫu ở cuối luồng xử lý** (tail-based sampling), cơ chế này được minh họa trong Hình 1.2; Thay vì quyết định tại thời điểm trace bắt đầu, phương pháp này chờ đến khi thu thập được toàn bộ hoặc phần lớn các span của trace, sau đó mới đánh giá trace đó có nên được giữ lại hay không [9, 10]. Nhờ có đầy đủ thông tin sau khi yêu cầu kết thúc, phương pháp lấy mẫu ở cuối luồng xử lý có khả năng giữ lại nhiều trace hữu ích hơn cho việc chẩn đoán lỗi so với các phương pháp lấy mẫu ở đầu luồng xử lý. Tuy nhiên, nhược điểm của hướng tiếp cận này là hệ thống vẫn phải thu thập đầy đủ và xử lý trace trước khi quyết định loại bỏ. Điều này làm tăng chi phí về mặt bộ nhớ và xử lý tại bộ thu thập dữ liệu đồng thời tăng độ phức tạp khi triển khai trong môi trường phân tán, đặc biệt khi cần đảm bảo các span thuộc cùng một trace được gom về cùng một điểm xử lý [10, 12, 15].

Trong khuôn khổ tail-based sampling, một số nghiên cứu đề xuất phương pháp **lấy mẫu định hướng** (directed sampling) nhằm giảm lãng phí tài nguyên lưu trữ bằng cách ưu tiên các trace hiếm hoặc bất thường, tiêu biểu như **Sifter** [13] và **Sieve** [16]. Mặc dù tối ưu được không gian lưu trữ, cách tiếp cận này bộc lộ nhiều hạn chế khi ứng dụng vào bài toán Phân tích nguyên nhân gốc. Cụ thể, các thuật toán RCA hiệu quả đòi hỏi cơ sở đối chiếu liên tục giữa trace lỗi và trace bình thường, đồng thời phải duy trì được các mối quan hệ phụ thuộc quan trọng giữa các dịch vụ [3, 7]. Do đó, nếu chiến lược lấy mẫu bị thiên lệch quá lớn về nhóm

trace bất thường, hệ thống sẽ đánh mất ngữ cảnh đối chiếu toàn cục, dẫn đến sự suy giảm trực tiếp về chất lượng và độ chính xác của bước phân tích lỗi.

Một hướng tiếp cận tiên tiến hơn là kết hợp thông tin trace với các chỉ số trạng thái hệ thống, tiêu biểu như **TraStrainer** [14]. Phương pháp này sử dụng cơ chế bỏ phiếu động (**dynamic voting**) để tích hợp thiên lệch hệ thống (**system-bias**) và thiên lệch đa dạng (**diversity-bias**), qua đó tối ưu hóa việc giữ lại các trace có giá trị nhất cho RCA. Mặc dù vậy, hướng tiếp cận này cũng đặt ra một số ràng buộc khi triển khai thực tế. Cụ thể, phương pháp cần các thành phần biểu diễn trace, xử lý hoặc dự báo metric, đồng thời phải đồng bộ dữ liệu giữa hệ thống truy vết như **Jaeger/Zipkin** và hệ thống giám sát hạ tầng theo thời gian thực. Điều này làm tăng độ phức tạp của pipeline lấy mẫu, đặc biệt trong các môi trường yêu cầu phản hồi nhanh, tài nguyên giới hạn hoặc tải thay đổi liên tục [14, 17, 18].

Song song với các nghiên cứu về phương pháp lấy mẫu, bài toán xác định nguyên nhân gốc rễ trong hệ thống vi dịch vụ cũng đã được nghiên cứu rộng rãi. Một nhóm phương pháp tiêu biểu là các phương pháp dựa trên đồ thị. Trong nhóm này, **MicroRank** tận dụng dữ liệu tracing để phát hiện các trace bất thường, xây dựng quan hệ giữa các thể hiện dịch vụ (service instance), sau đó kết hợp **PageRank** với định vị lỗi dựa trên phổ thực thi (spectrum-based fault localization – SBFL) để xếp hạng các dịch vụ có khả năng gây ra lỗi [3]. Thay vì chỉ đếm số lần lỗi xuất hiện tại từng dịch vụ, **MicroRank** xem xét cả cấu trúc phụ thuộc và sự khác biệt giữa trace bình thường và trace bất thường, từ đó đưa ra danh sách nghi vấn có ý nghĩa hơn cho các hệ thống giám sát.

Ngoài các phương pháp dựa trên đồ thị, một số hướng tiếp cận khác tập trung vào phân cụm trace hoặc phát hiện bất thường trong trace [8, 11]. Các phương pháp này có thể nhận diện những trace có cấu trúc, độ trễ hoặc mẫu thực thi khác biệt so với hành vi bình thường. Tuy nhiên, khi áp dụng vào RCA, việc phát hiện trace bất thường vẫn chưa đủ để xác định dịch vụ gây lỗi, do cần thêm bước suy luận để chuyển từ bất thường ở mức trace sang nguyên nhân gốc ở mức dịch vụ [3, 7].

Nhìn chung, các nghiên cứu hiện tại bộc lộ một sự đánh đổi lớn giữa chi phí lấy mẫu và chất lượng dữ liệu phục vụ bài toán phân tích nguyên nhân gốc. Các phương pháp lấy mẫu cơ bản tiết kiệm chi phí nhưng không đảm bảo đủ trace chất lượng hoặc làm mất cân bằng tỉ lệ giữa dữ liệu lỗi và dữ liệu bình thường, vốn là cơ sở then chốt cho các thuật toán RCA đối chiếu [3, 9]. Mặt khác, các phương pháp lấy

mẫu thích ứng tuy hiệu quả nhưng thường kéo theo chi phí vận hành cao do phụ thuộc vào các mô hình học máy phức tạp [14, 17, 18].

Khoảng trống này gợi ý một hướng tiếp cận khác, một chiến lược lấy mẫu tinh gọn, không giám sát (unsupervised) và hoạt động theo thời gian thực. Bằng cách khai thác trực tiếp các tín hiệu nguyên bản của trace như lỗi, độ trễ, tần suất và quan hệ phụ thuộc, hướng tiếp cận này vừa tối ưu chi phí lưu trữ, vừa duy trì tốt tính phân biệt giữa hành vi bình thường và bất thường, qua đó hỗ trợ tối đa cho các thuật toán RCA phía sau [3, 7, 14].

1.3 Mục tiêu và đóng góp của khoá luận

Xuất phát từ nhu cầu giảm chi phí lưu trữ trace nhưng vẫn bảo đảm chất lượng dữ liệu cho bài toán xác định nguyên nhân gốc, khoá luận này đề xuất một phương pháp lấy mẫu thích ứng cho hệ thống vi dịch vụ. Khác với các phương pháp lấy mẫu ngẫu nhiên hoặc theo tỉ lệ cố định, phương pháp này đánh giá mức độ quan trọng của từng trace dựa trên các tín hiệu sẵn có trong quá trình vận hành, bao gồm trạng thái bất thường của hệ thống, độ đa dạng của trace và độ gần với thời điểm xảy ra sự cố. Từ đó, hệ thống ưu tiên giữ lại những trace có nhiều khả năng hữu ích cho phân tích lỗi [3, 14].

Mục tiêu thiết kế cốt lõi là một cơ chế lấy mẫu có thể hoạt động dưới ràng buộc ngân sách, không phụ thuộc vào dữ liệu gán nhãn và không yêu cầu huấn luyện giám sát phức tạp. Để đạt được điều này, phương pháp sử dụng cơ chế chấm điểm theo luật kinh nghiệm (heuristic scoring) để kết hợp các tín hiệu từ trace và trạng thái hệ thống, qua đó đưa ra quyết định giữ hoặc loại bỏ trace theo hướng thích ứng với bối cảnh hiện tại. Cách tiếp cận này không đòi hỏi pipeline metric phức tạp như các phương pháp học máy, trong khi vẫn duy trì được tính phân biệt giữa trace bình thường và trace bất thường cần thiết cho các thuật toán RCA phía sau [17, 18].

Bên cạnh việc thiết kế bộ lấy mẫu, khoá luận còn tích hợp phương pháp đề xuất với MicroRank để đánh giá trực tiếp ảnh hưởng của việc lấy mẫu tới chất lượng RCA [3]. Cụ thể, các trace sau khi được lấy mẫu sẽ được chuyển đổi thành đầu vào cho MicroRank để xếp hạng các dịch vụ nghi vấn là nguyên nhân gốc. Kết quả được đánh giá bằng các độ đo **A@1**, **A@3** và **MRR**. Cách đánh giá này phản ánh

sát hơn mục tiêu thực tế của bài toán, vì một phương pháp lấy mẫu chỉ thực sự có giá trị khi dữ liệu được giữ lại vẫn duy trì hoặc cải thiện chất lượng RCA dưới ngân sách nhỏ [7, 14].

Để đánh giá phương pháp, khoá luận thực hiện thực nghiệm trên **TraStrainer Dataset B**, gồm các hệ thống **train-ticket**, **hipster-batch1** và **hipster-batch2** [14]. Phương pháp đề xuất được so sánh với các hướng tiếp cận tiêu biểu như **Random**, **Sieve**, **Sifter** và **TraStrainer** trên cùng bộ dữ liệu, cùng ngân sách lấy mẫu và cùng bộ độ đo đánh giá. Qua đó, khoá luận phân tích khả năng giữ lại trace hữu ích cho RCA cũng như sự đánh đổi giữa ngân sách lưu trữ, chất lượng xếp hạng nguyên nhân gốc và chi phí xử lý dữ liệu [3].

Để thực hiện đánh giá một cách có hệ thống, khoá luận tập trung trả lời các câu hỏi nghiên cứu sau:

- **RQ1: So sánh hiệu suất.** Phương pháp lấy mẫu thích ứng được đề xuất hỗ trợ bài toán RCA hiệu quả như thế nào khi so sánh với các phương pháp lấy mẫu hiện có như **Random**, **Sieve**, **Sifter** và **TraStrainer**?
- **RQ2: Ảnh hưởng của ngân sách lấy mẫu.** Khi ngân sách lưu trữ thay đổi, chất lượng RCA sau lấy mẫu biến động như thế nào? Phương pháp được đề xuất có duy trì được hiệu quả trong điều kiện ngân sách rất nhỏ hay không?
- **RQ3: Phân tích đóng góp của từng thành phần.** Các tín hiệu trong cơ chế chấm điểm theo luật kinh nghiệm, bao gồm tín hiệu bất thường của hệ thống, độ đa dạng của trace và độ gần với thời điểm xảy ra sự cố, đóng góp như thế nào vào chất lượng lấy mẫu và kết quả RCA cuối cùng?
- **RQ4: Phân tích sự đánh đổi (trade-off).** Phương pháp được đề xuất tạo ra sự đánh đổi như thế nào giữa chi phí lưu trữ, chi phí xử lý và chất lượng xác định nguyên nhân gốc?

Thông qua việc giải quyết các bài toán và câu hỏi nghiên cứu đã đặt ra, những đóng góp cốt lõi của khóa luận này bao gồm:

- Đề xuất một phương pháp lấy mẫu thích ứng dạng luồng (streaming) cho dữ liệu tracing trong hệ thống vi dịch vụ, dựa trên cơ chế chấm điểm theo luật kinh nghiệm kết hợp các tín hiệu về trạng thái bất thường hệ thống, độ đa

dạng trace và độ gần với sự cố. Phương pháp không phụ thuộc vào huấn luyện giám sát và có thể hoạt động theo thời gian thực.

- Xây dựng pipeline đánh giá đầu-cuối (end-to-end) từ lấy mẫu thích nghi, chuyển đổi dữ liệu trace sau lấy mẫu thành đầu vào cho MicroRank, thực thi MicroRank RCA, đến đánh giá kết quả xếp hạng bằng các độ đo thống nhất.
- Thực nghiệm và so sánh phương pháp được đề xuất với các phương pháp lấy mẫu tiêu biểu như **Random**, **Sieve**, **Sifter** và **TraStrainer** trên **TraStrainer Dataset B**, bao gồm *train-ticket*, *hipster-batch1* và *hipster-batch2*.
- Phân tích sự đánh đổi giữa ngân sách lấy mẫu, chất lượng RCA và chi phí lưu trữ, qua đó làm rõ khả năng ứng dụng của phương pháp trong các hệ thống vi dịch vụ quy mô lớn.

1.4 Cấu trúc khóa luận

Trong những phần còn lại, khoá luận sẽ đi sâu vào các nội dung sau:

Chương 2 trình bày các kiến thức nền tảng về kiến trúc vi dịch vụ, theo dõi phân tán, lấy mẫu trace và bài toán phân tích nguyên nhân gốc trong hệ thống phân tán. Đồng thời cung cấp thông tin về các kỹ thuật lấy mẫu trace và các thành phần được sử dụng trong phương pháp lấy mẫu thích nghi đề xuất.

Chương 3 đi sâu vào phương pháp lấy mẫu thích nghi, bao gồm biểu diễn trace, cơ chế chấm điểm theo luật kinh nghiệm, lấy mẫu dạng luồng và quá trình chuyển đổi dữ liệu sau lấy mẫu thành đầu vào cho pipeline RCA. Sau đó mô tả phiên bản mở rộng kết hợp trace với metric hệ thống để cải thiện chất lượng tập trace được giữ lại và kết quả RCA bằng **MicroRank**.

Chương 4 tập trung vào thiết lập thực nghiệm, bao gồm mô tả bộ dữ liệu, các phương pháp so sánh, độ đo đánh giá. Kết quả thực nghiệm được phân tích nhằm đánh giá hiệu quả của từng thành phần, đồng thời làm rõ sự đánh đổi (trade-off) giữa ngân sách lấy mẫu và chất lượng định vị nguyên nhân gốc.

Phần Kết luận tổng kết lại những đóng góp và kết quả chính của khoá luận. Từ đó, khoá luận cũng chỉ ra một số hạn chế còn tồn tại của phương pháp và đề xuất các hướng nghiên cứu, phát triển tiếp theo.

Chương 2

Cơ sở lý thuyết và nền tảng bài toán

Trong hệ thống vi dịch vụ, một yêu cầu nghiệp vụ thường đi qua nhiều dịch vụ độc lập. Khi sự cố xảy ra, lỗi có thể lan truyền qua nhiều thành phần, khiến việc xác định dịch vụ gây lỗi trở nên khó khăn [19, 20]. Dữ liệu truy vết phân tán cung cấp góc nhìn quan trọng về quá trình xử lý khi gặp lỗi trong hệ thống. Tuy nhiên, khối lượng trace sinh ra trong thực tế rất lớn, làm tăng chi phí lưu trữ và xử lý. Vì vậy, lấy mẫu trace dưới ràng buộc ngân sách là hướng tiếp cận cần thiết [9, 17].. Trong khóa luận này, mục tiêu lấy mẫu không chỉ là giảm dữ liệu hay giữ trace bất thường, mà là giữ lại tập trace có ích cho RCA ở bước sau. Các khái niệm trong chương này là cơ sở để xây dựng phương pháp lấy mẫu thích nghi được trình bày trong Chương 3.

2.1 Hệ thống vi dịch vụ và dữ liệu quan sát

2.1.1 Kiến trúc hệ thống vi dịch vụ

Kiến trúc vi dịch vụ là phong cách thiết kế trong đó ứng dụng được chia thành nhiều dịch vụ nhỏ, mỗi dịch vụ đảm nhận một chức năng nghiệp vụ tương đối độc lập. Khác với kiến trúc nguyên khối, các dịch vụ trong hệ thống vi dịch vụ có thể được phát triển, triển khai và mở rộng độc lập [4, 5].

Mỗi dịch vụ thường có phạm vi trách nhiệm rõ ràng và giao tiếp với các dịch vụ khác thông qua mạng, ví dụ **HTTP/REST**, **gRPC**, hàng đợi thông điệp (message queue) hoặc luồng sự kiện (event streaming). Cách tổ chức này giúp hệ thống dễ mở rộng và phù hợp với mô hình phát triển theo nhóm nhỏ. Tuy nhiên, nó cũng làm tăng độ phức tạp vận hành. Một yêu cầu có thể đi qua nhiều dịch vụ; lỗi ở một dịch vụ có thể gây quá thời gian quy định (timeout), tăng độ trễ hoặc lỗi thứ cấp ở các dịch vụ phía sau [5]. Vì vậy, Việc gỡ lỗi bằng log cục bộ của từng dịch vụ thường không đủ. Người vận hành cần một cơ chế quan sát toàn cục để biết yêu cầu đã đi qua những thành phần nào, thời gian xử lý ở từng thành phần ra sao và lỗi bắt đầu xuất hiện từ đâu. Đây chính là lý do tại sao khả năng quan

sát (observability), đặc biệt là truy vết phân tán, đã trở thành một thành phần cốt lõi trong hệ sinh thái vi dịch vụ.

2.1.2 Dữ liệu quan sát trong hệ thống phân tán

Khả năng quan sát hay khả năng giám sát hệ thống (observability) là khả năng suy luận trạng thái bên trong của hệ thống thông qua các tín hiệu được thu thập trong quá trình vận hành. Trong hệ thống phân tán, khả năng quan sát thường dựa trên ba nhóm dữ liệu chính: logs, metrics và traces [21].

- **Logs:** ghi lại các sự kiện rời rạc trong từng thành phần, chẳng hạn yêu cầu được tiếp nhận, truy vấn cơ sở dữ liệu thất bại hoặc exception trong mã nguồn.
- **Metrics:** là các số liệu định lượng theo thời gian, ví dụ CPU, bộ nhớ, thông lượng (throughput), độ trễ (latency), tỷ lệ lỗi (error rate) hoặc số lượng yêu cầu trên giây.
- **Traces:** mô tả đường đi của một yêu cầu qua nhiều dịch vụ, cho biết request đi qua dịch vụ nào, thứ tự gọi giữa các dịch vụ và vị trí phát sinh lỗi nếu có.

Trong thực tế, các nền tảng quan sát hiện đại thường kết hợp cả ba loại dữ liệu. **OpenTelemetry** là một bộ tiêu chuẩn và công cụ mã nguồn mở, trung lập với nhà cung cấp, hỗ trợ thu thập, sinh, xử lý và xuất dữ liệu telemetry như logs, metrics và traces [21]. Trong khóa luận này, trace là nguồn dữ liệu chính phục vụ lấy mẫu và RCA, trong khi metrics đóng vai trò hỗ trợ để đánh giá trạng thái hệ thống và xác định được sự quan trọng của traces trong từng thời điểm đặc biệt của hệ thống.

2.1.3 Truy vết phân tán

Truy vết phân tán là kỹ thuật theo dõi quá trình xử lý của một yêu cầu khi yêu cầu đó đi qua nhiều thành phần trong hệ thống phân tán. Mục tiêu là tái dựng đường đi của yêu cầu, từ đó giúp xác định dịch vụ nào tham gia xử lý, thao tác nào tốn thời gian và lỗi xuất hiện ở đâu [6, 12].

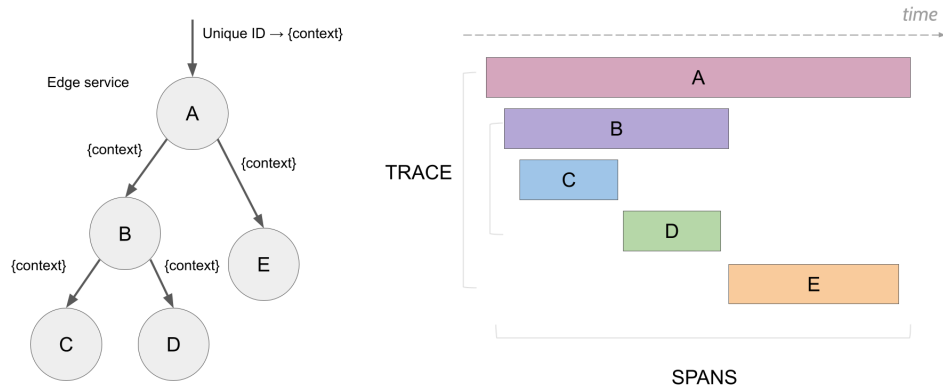
Một trace biểu diễn toàn bộ hành trình của một yêu cầu và được cấu thành từ nhiều span. Mỗi span thường gắn với một operation, tức thao tác cụ thể được thực

hiện trong quá trình xử lý yêu cầu, chẳng hạn một lời gọi API, một truy vấn cơ sở dữ liệu hoặc một lời gọi sang dịch vụ khác. Một span thường chứa các thông tin như `trace_id`, `span_id`, `parent_span_id`, dịch vụ sinh ra span, operation, thời điểm bắt đầu, thời lượng xử lý và trạng thái lỗi [21].

Nhờ quan hệ cha-con giữa các span, trace có thể được biểu diễn dưới dạng cây gọi. Từ đó có thể suy ra thứ tự xử lý các yêu cầu, nhánh gọi song song hoặc tuần tự, đường đi qua các dịch vụ và đồ thị phụ thuộc dịch vụ. Đây là nguồn thông tin quan trọng cho RCA, vì nó gắn tín hiệu lỗi và độ trễ với ngữ cảnh thực thi của yêu cầu [3, 6].

2.2 Dữ liệu truy vết phân tán

2.2.1 Span và trace



Hình 2.1: Mối quan hệ giữa span và trace, cấu trúc topology của trace [2].

Span là đơn vị dữ liệu cơ bản trong truy vết phân tán. Mỗi span mô tả một thao tác cụ thể trong quá trình xử lý yêu cầu, các span có cùng `trace_id` được gom lại thành một trace [6, 21]. Do đó, một trace có thể được xem như bản ghi đầy đủ về hành trình xử lý của một yêu cầu.

Ở mức dữ liệu thô, trace thường chứa nhiều span với thông tin chi tiết. Tuy nhiên, trong nhiều bài toán phân tích và RCA, dữ liệu span có thể được tổng hợp thành biểu diễn ở mức trace để giảm độ phức tạp và phục vụ các bước phân tích tiếp theo [3, 7]. Biểu diễn mức trace thường giữ lại các đặc trưng như thời điểm bắt đầu, thời lượng, số lượng span, tập dịch vụ xuất hiện trong trace và trạng thái lỗi.

2.2.2 Cấu trúc topo của trace

Trace không chỉ là một tập span độc lập, mà còn chứa cấu trúc **topo** (topology), tức cấu trúc quan hệ cha-con giữa các span trong quá trình xử lý yêu cầu. Như minh họa trong Hình 2.1, quan hệ này cho phép biểu diễn trace dưới dạng cây hoặc đồ thị có hướng [6, 12]. Từ đó cho biết dịch vụ nào gọi tới dịch vụ nào, lời gọi nào xảy ra trước và yêu cầu đi qua những đường xử lý nào.

Cấu trúc **topo** có ý nghĩa quan trọng trong phân tích lỗi. Nếu một dịch vụ ở gần đầu chuỗi xử lý gặp sự cố, lỗi hoặc độ trễ có thể lan truyền tới nhiều dịch vụ phía sau. Các phương pháp RCA dựa trên trace, như **MicroRank**, khai thác thông tin xuất hiện của dịch vụ trong các trace bình thường và bất thường để xếp hạng dịch vụ gây nguyên nhân gốc [3]. Trong bài toán lấy mẫu, tập dịch và đường dẫn dịch vụ (service path) còn có thể được dùng để đo độ tương đồng hoặc độ đa dạng giữa các trace. Cũng chính là chỉ số Jaccard ở phần sau.

2.2.3 Các đặc trưng của trace

Để phục vụ lấy mẫu và RCA, dữ liệu trace thường được chuyển thành tập đặc trưng ở mức trace. Các đặc trưng như thời điểm bắt đầu, thời lượng, trạng thái lỗi, số lượng span và tập service xuất hiện trong trace thường được khai thác trong các phương pháp phân tích trace và định vị nguyên nhân gốc [3, 7, 14]. Trong khóa luận này, các đặc trưng quan trọng gồm:

- **trace_id**: định danh duy nhất của trace;
- **start_ns**: thời điểm bắt đầu trace;
- **duration**: thời gian xử lý của trace;
- **span_count**: số lượng span trong trace;
- **has_error**: trace có chứa lỗi hay không;
- **services**: tập service xuất hiện trong trace;
- **service_path**: đường đi của request qua các service, nếu có;
- **root_service**: service bắt đầu hoặc service gốc của request, nếu có.

Các đặc trưng này phản ánh những khía cạnh khác nhau của trace. Ví dụ như `duration` liên quan đến tín hiệu độ trễ, `span_count` phản ánh độ phức tạp, `has_error` biểu diễn tín hiệu lỗi, còn `services` và `service_path` cho biết trace có đi qua các service liên quan đến sự cố hay không. Ngoài các đặc trưng trực tiếp từ trace, một số phương pháp gần đây còn kết hợp tín hiệu trạng thái hệ thống như độ trễ (`latency`), tải công việc (`workload`) hoặc metric vận hành theo thời gian thực để cải thiện chất lượng lựa chọn trace [14].

2.2.4 Đặc điểm và thách thức của dữ liệu trace

Dữ liệu trace trong hệ thống vi dịch vụ có khối lượng lớn, tốc độ sinh cao và thường chứa nhiều trace có cấu trúc lặp lại. Các nghiên cứu về truy vết và lấy mẫu trace đều nhấn mạnh rằng việc thu thập toàn bộ trace trong hệ thống lớn có thể gây chi phí cao, do đó cần các cơ chế lựa chọn hoặc lấy mẫu phù hợp [12–14].

Một thách thức khác là dữ liệu trace thường mất cân bằng. Phần lớn trace là trace bình thường, trong khi trace lỗi hoặc trace liên quan trực tiếp đến sự cố chiếm tỷ lệ nhỏ. Nếu lấy mẫu ngẫu nhiên với tỷ lệ thấp, các trace quan trọng có thể bị bỏ lỡ. Ngược lại, nếu quá ưu tiên trace lỗi, sampler có thể bỏ qua trace bình thường cần thiết cho đối sánh RCA.

Ngoài ra, trace lỗi không phải lúc nào cũng hữu ích cho RCA. Một trace lỗi có thể phản ánh lỗi thứ cấp hoặc lỗi lan truyền, trong khi trace bình thường vẫn có thể cung cấp nền đối sánh cần thiết cho các phương pháp RCA dựa trên phổ lỗi [3, 14]. Vì vậy, bài toán lấy mẫu trace không đơn thuần là giảm kích thước dữ liệu, giữ lại trace lỗi, mà là giữ lại một tập trace nhỏ nhưng giàu thông tin cho RCA.

2.3 Phân tích nguyên nhân gốc rễ trong hệ thống phân tán

2.3.1 Khái niệm phân tích nguyên nhân gốc rễ

Phân tích nguyên nhân gốc rễ là quá trình xác định thành phần hoặc sự kiện ban đầu gây ra một sự cố quan sát được trong hệ thống. Trong hệ thống vi dịch vụ, RCA thường nhằm tìm ra dịch vụ, điểm cuối (endpoint) hoặc thao tác có khả năng là nguyên nhân chính dẫn đến lỗi, độ trễ tăng cao hoặc suy giảm chất lượng dịch vụ [7, 11].

Khác với phát hiện bất thường, RCA không chỉ xác định rằng hệ thống đang có vấn đề, mà còn cần chỉ ra thành phần nào có khả năng gây ra vấn đề đó. Đầu vào của RCA có thể gồm logs, metrics, traces, đồ thị phụ thuộc dịch vụ và thông tin của sổ sự cố. Đầu ra thường là danh sách service được xếp hạng theo mức độ nghi ngờ, giúp người vận hành ưu tiên kiểm tra một nhóm service nhỏ thay vì toàn bộ hệ thống [3].

Trong khóa luận này, RCA được xem như bước xử lý phía sau bước lấy mẫu (sampler). Quá trình lấy mẫu chọn một tập trace nhỏ từ dữ liệu gốc; sau đó tập trace này được đưa vào phương pháp RCA để sinh ra bảng xếp hạng các dịch vụ. Vì vậy, chất lượng RCA phụ thuộc trực tiếp vào chất lượng tập trace được giữ lại.

2.3.2 RCA dựa trên phổ lỗi

Một hướng tiếp cận phổ biến trong định vị lỗi (fault localization) là phân tích dựa trên phổ lỗi. Ý tưởng cốt lõi là đánh giá mức độ đáng nghi của một thành phần dựa trên tần suất thành phần đó xuất hiện trong các trường hợp lỗi và trường hợp bình thường [3, 22].

	Trace có chứa dịch vụ	Trace không chứa dịch vụ
Trace bất thường	n_{ef}	n_{nf}
Trace bình thường	n_{ep}	n_{np}

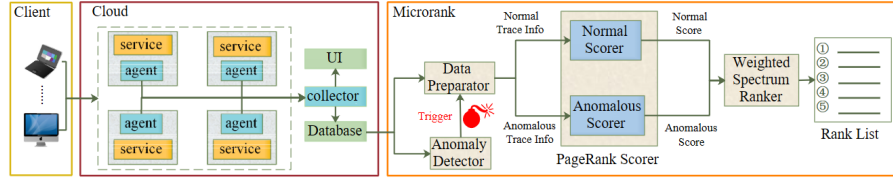
Bảng 2.1: Ma trận thống kê sự xuất hiện của một dịch vụ trong các trace bất thường và bình thường.

Trong ngữ cảnh trace, một dịch vụ có thể được xem như một thành phần. Với mỗi dịch vụ, các thống kê trong Bảng 2.1 cho biết số trace bất thường có chứa dịch vụ, số trace bất thường không chứa dịch vụ và số trace bình thường không chứa dịch vụ. Nếu một dịch vụ xuất hiện nhiều trong trace bất thường nhưng ít xuất hiện trong trace bình thường, dịch vụ đó có thể có mức nghi ngờ cao.

Điểm quan trọng của RCA dựa trên phổ lỗi là nó cần cả trace bất thường và trace bình thường. Trace bất thường cung cấp tín hiệu về sự cố, còn trace bình thường cung cấp nền đối chứng. Nếu tập trace sau lấy mẫu chỉ gồm trace lỗi, phương pháp RCA có thể thiếu cơ sở để phân biệt service thực sự đáng nghi với service phổ

biến. Đây là cơ sở lý thuyết cho yêu cầu cân bằng giữa trace bất thường và trace nền trong bài toán lấy mẫu phục vụ RCA.

2.3.3 MicroRank



Hình 2.2: Minh hoạ cấu trúc microrank [3].

MicroRank là một phương pháp RCA trong hệ thống vi dịch vụ dựa trên dữ liệu trace. Như minh hoạ trong Hình 2.2, phương pháp này sử dụng mở rộng của phân tích phổ lỗi để khai thác thông tin từ trace bình thường (normal traces) và trace bất thường (abnormal traces), từ đó định vị các service có khả năng gây ra vấn đề độ trễ trong môi trường microservice [3].

Ở mức khái quát, **MicroRank** xây dựng quan hệ giữa trace và dịch vụ, sau đó tính mức độ đáng nghi của từng service dựa trên sự khác biệt giữa trace bất thường và trace bình thường. Chất lượng đầu vào có ảnh hưởng trực tiếp đến kết quả của **MicroRank**. Nếu bộ lấy mẫu làm mất trace đi qua dịch vụ gây ra nguyên nhân gốc, hoặc làm tập trace bất thường chứa quá nhiều nhiễu, kết quả xếp hạng dịch vụ có thể bị sai lệch [3, 14].

Vì vậy, **MicroRank** không chỉ là phương pháp RCA được dùng ở bước sau, mà còn là một ràng buộc thiết kế đối với bộ lấy mẫu. Mục tiêu không phải là chọn các trace có điểm bất thường cao nhất, mà là chọn tập trace giúp **MicroRank** xếp hạng đúng danh sách dịch vụ gây ra nguyên nhân gốc.

2.3.4 Các độ đo đánh giá chất lượng RCA

Để đánh giá chất lượng RCA, nhiều nghiên cứu sử dụng các độ đo dựa trên vị trí của dịch vụ nguyên nhân gốc trong bảng xếp hạng, chẳng hạn **A@k** và **MRR** [3, 7, 14]. Trong khóa luận này, ba độ đo chính gồm **A@1**, **A@3** và **MRR**.

Với N kịch bản sự cố (scenario) và $rank_i$ là thứ hạng của dịch vụ nguyên nhân

gốc trong kịch bản thứ i , **A@1** được định nghĩa:

$$\mathbf{A@1} = \frac{1}{N} \sum_{i=1}^N \mathbb{I}(\text{rank}_i = 1). \quad (2.1)$$

Theo Công thức (2.1), **A@1** đo tỷ lệ trường hợp trong đó nguyên nhân gốc được xếp ở vị trí đầu tiên.

A@3 đo tỷ lệ trường hợp nguyên nhân gốc nằm trong ba vị trí đầu:

$$\mathbf{A@3} = \frac{1}{N} \sum_{i=1}^N \mathbb{I}(\text{rank}_i \leq 3). \quad (2.2)$$

Theo Công thức (2.2), độ đo này phù hợp với bối cảnh vận hành, khi người vận hành có thể kiểm tra một nhóm nhỏ các dịch vụ đáng nghi.

MRR đo chất lượng thứ hạng tổng thể:

$$\mathbf{MRR} = \frac{1}{N} \sum_{i=1}^N \frac{1}{\text{rank}_i}. \quad (2.3)$$

Theo Công thức (2.3), nguyên nhân gốc càng đứng cao thì **MRR** càng lớn. Các độ đo này được sử dụng để đánh giá tác động của chiến lược sampling lên chất lượng RCA [3].

2.4 Bài toán lấy mẫu trace phục vụ RCA

2.4.1 Lấy mẫu trace dưới ràng buộc ngân sách

Trong hệ thống vi dịch vụ có lưu lượng lớn, việc lưu trữ và xử lý toàn bộ trace là không khả thi hoặc không đáp ứng được yêu cầu về chi phí. Vì vậy, các hệ thống truy vết thường hỗ trợ cơ chế lấy mẫu để giới hạn lượng trace được thu thập và xử lý [9, 10, 13].

Gọi $\mathcal{T}$ là tập trace đầy đủ và $\mathcal{S}$ là tập trace được giữ lại. Với ngân sách lấy mẫu B , bài toán lấy mẫu có thể được mô tả bởi công thức (2.4):

$$\mathcal{S} \subseteq \mathcal{T}, \quad |\mathcal{S}| \leq B \cdot |\mathcal{T}|. \quad (2.4)$$

Ví dụ, với $B = 0.001$, bộ lấy mẫu chỉ được giữ lại khoảng 0.1% tổng số trace.

Ràng buộc về mặt ngân sách tạo ra sự đánh đổi giữa chi phí và chất lượng phân tích. Nếu giữ quá ít trace, RCA có thể thiếu thông tin để xác định nguyên nhân gốc. Nếu giữ quá nhiều trace, chi phí lưu trữ và xử lý tăng lên. Do đó, bộ xử lý cần chọn tập trace nhỏ nhưng vẫn đủ thông tin cho RCA.

2.4.2 Lấy mẫu theo dòng

Lấy mẫu theo dòng (streaming sampling) là bối cảnh trong đó trace xuất hiện liên tục theo thời gian và bộ lấy mẫu phải quyết định việc giữ hoặc bỏ trace khi trace được quan sát. Các nghiên cứu gần đây về lấy mẫu trace nhấn mạnh yêu cầu xử lý trực tuyến (online), khả năng mở rộng và thích nghi với trạng thái hệ thống khi lựa chọn trace [14, 17, 18].

Ràng buộc theo dòng khiến bộ lấy mẫu chỉ có thể sử dụng thông tin hiện tại và lịch sử trước đó. Các thống kê như độ trễ trung bình, tỷ lệ lỗi hoặc phân phối thời lượng xử lý cần được cập nhật trực tuyến. Ngoài ra, bộ lấy mẫu cần giới hạn bộ nhớ và xử lý nhanh trên từng trace để phù hợp với môi trường vận hành gần thời gian thực.

Lấy mẫu theo dòng khó hơn lấy mẫu ngoại tuyến (offline sampling) vì nhiều tín hiệu quan trọng chỉ rõ ràng sau khi sự cố diễn ra một thời gian. Một trace đến sớm trong cửa sổ sự cố có thể chưa được nhận diện là bất thường do hệ thống chưa tích lũy đủ lịch sử. Vì vậy, phương pháp lấy mẫu cần cân bằng giữa phản ứng nhanh và độ tin cậy của tín hiệu [14, 18].

2.4.3 Mục tiêu lấy mẫu nhận biết RCA

Lấy mẫu trace thông thường thường hướng tới mục tiêu giảm dữ liệu hoặc giữ lại trace có độ trễ cao, có lỗi, hoặc có điểm bất thường lớn. Tuy nhiên, trong bài toán lấy mẫu phục vụ RCA, mục tiêu quan trọng hơn là giữ lại tập trace giúp RCA xếp hạng đúng dịch vụ gây ra nguyên nhân gốc. Bộ phân tích RCA ở bước sau cần cả tín hiệu bất thường và dữ liệu nền để tạo cơ sở đối sánh [3, 14, 23].

Mục tiêu lấy mẫu có thể được diễn đạt theo công thức (2.5) như sau:

$$\mathcal{S}^* = \arg \max_{\mathcal{S} \subseteq \mathcal{T}} Q_{RCA}(\mathcal{S}) \quad \text{với} \quad |\mathcal{S}| \leq B \cdot |\mathcal{T}|, \quad (2.5)$$

Trong đó $Q_{RCA}(\mathcal{S})$ là chất lượng RCA đạt được khi sử dụng tập trace $\mathcal{S}$.

Trong thực tế, Q_{RCA} không thể tính trực tiếp tại thời điểm lấy mẫu vì cần chạy RCA và biết nhân nguyên nhân gốc. Vì vậy, bộ lấy mẫu cần dùng các tín hiệu thay thế như lỗi, độ trễ, service path, độ gần cửa sổ sự cố, độ đa dạng và tín hiệu metric để ước lượng mức độ hữu ích của trace. Điểm cốt lõi là bài toán không phải chọn top- k trace bất thường nhất, mà là chọn tập trace có ích nhất cho bộ phân tích RCA ở giai đoạn sau.

2.4.4 Ảnh hưởng của sampling đến chất lượng RCA

Do RCA chỉ được thực hiện trên tập trace sau khi lấy mẫu, chất lượng lấy mẫu ảnh hưởng trực tiếp đến chất lượng xếp hạng. Các phương pháp như **MicroRank** phụ thuộc vào thông tin từ tập trace bình thường và bất thường, trong khi các phương pháp lấy mẫu hiện đại nhấn mạnh rằng trace được giữ lại cần vừa có tính đại diện, vừa chứa tín hiệu bất thường có ích [3, 13, 14].

Một số dạng lỗi thường gặp gồm: thiếu trace chứa dịch vụ nguyên nhân gốc, giữ quá nhiều trace lỗi không liên quan, thiếu trace bình thường làm mất nền đối chiếu, hoặc giữ quá nhiều trace trùng lặp. Các trường hợp này đều có thể làm việc xếp hạng bị lệch, dù số lượng trace được giữ lại vẫn đúng theo ngân sách. Vì vậy, thiết kế sampler cần được đánh giá bằng chất lượng RCA cuối cùng, không chỉ bằng các chỉ số nội bộ của sampling.

2.4.5 Thách thức ở mức ngân sách thấp

Ngân sách thấp là thách thức lớn vì chỉ một số lượng rất nhỏ trace được giữ lại, trong khi các trace quan trọng cho chẩn đoán có thể xuất hiện với tần suất thấp. Các phương pháp như **Sifter**, **Sieve** và **TraStrainer** tiếp cận vấn đề này bằng cách ưu tiên các trace có giá trị thông tin cao hơn so với lấy mẫu thuần túy [13, 14, 16].

Ở mức ngân sách như 0.1%, chỉ một vài quyết định giữ hoặc bỏ trace cũng có thể làm thay đổi đáng kể kết quả RCA. Ngoài ra, tín hiệu bất thường có thể chưa ổn định ở giai đoạn đầu của sự cố, khiến các trace sớm nhưng quan trọng bị bỏ qua [3, 14]. Điều này tạo động lực cho các kỹ thuật như chấm điểm thích nghi (adaptive scoring), kiểm soát độ đa dạng (diversity control), kết hợp nhiều lần chạy (ensemble) và đồng thuận lựa chọn (consensus) để tăng độ ổn định của quá trình lấy mẫu.

2.4.6 Bài toán ổn định khi mở rộng ngân sách

Theo lý thuyết, khi ngân sách lấy mẫu tăng, chất lượng RCA nên được cải thiện vì hệ thống giữ lại nhiều trace hơn. Tuy nhiên, trong bài toán RCA, thêm nhiều trace chỉ có ích nếu các trace bổ sung cung cấp thông tin mới và phù hợp với cơ chế xếp hạng của bộ phân tích RCA. Các nghiên cứu gần đây về lấy mẫu trace cũng nhấn mạnh rằng độ đa dạng của trace, trạng thái hệ thống theo thời gian thực và tối ưu lựa chọn trace đều ảnh hưởng đến chất lượng tập trace được giữ lại [14, 18, 23].

Nếu phần trace được thêm chủ yếu trùng lặp với lõi trace đã chọn ở ngân sách thấp, tập dữ liệu lớn hơn nhưng không giàu thông tin hơn. Nếu phần trace được thêm chứa nhiều lỗi không liên quan hoặc phá vỡ cân bằng bình thường/bất thường, chất lượng RCA có thể giảm. Vì vậy, một bộ lấy mẫu tốt không chỉ cần mạnh ở điều kiện ngân sách thấp, mà còn cần mở rộng tập trace theo cách có kiểm soát khi ngân sách tăng.

2.5 Nền tảng kỹ thuật cho lựa chọn trace

2.5.1 Phát hiện bất thường trong dữ liệu hệ thống

Phát hiện bất thường là thành phần quan trọng trong nhiều hệ thống quan sát và giám sát. Trong bối cảnh vi dịch vụ, bất thường có thể xuất hiện dưới dạng độ trễ tăng, tỉ lệ lỗi tăng, thông lượng thay đổi hoặc sự biến động của các metric hệ thống [8, 11, 14].

Một cách đơn giản để phát hiện bất thường là so sánh giá trị hiện tại với thống kê lịch sử. Với x_t là giá trị tại thời điểm t , z-score được định nghĩa như trong công thức (2.6).

$$z_t = \frac{x_t - \mu}{\sigma}, \quad (2.6)$$

trong đó μ và σ là trung bình và độ lệch chuẩn của dữ liệu lịch sử. Trong môi trường có nhiễu, độ lệch tuyệt đối trung vị (median absolute deviation – MAD) có thể được dùng thay cho độ lệch chuẩn. MAD được định nghĩa như trong công thức (2.7).

$$MAD = \text{median}(|x_i - \text{median}(X)|). \quad (2.7)$$

Ngoài ra, trung bình động hàm mũ (exponential moving average – EMA) có thể

được dùng để cập nhật giá trị nền theo thời gian. EMA được định nghĩa như trong công thức (2.8).

$$EMA_t = \alpha x_t + (1 - \alpha)EMA_{t-1}. \quad (2.8)$$

Trong bài toán này, điểm bất thường (anomaly score) chỉ là một tín hiệu phụ. Một trace có độ trễ cao hoặc có lỗi chưa chắc là trace hữu ích nhất cho RCA. Vì vậy, phát hiện bất thường cần được kết hợp với tín hiệu RCA, độ đa dạng và ràng buộc ngân sách.

2.5.2 Đo lường độ tương đồng và đa dạng trace

Đa dạng trace là yếu tố quan trọng trong lấy mẫu dưới ràng buộc ngân sách. Các phương pháp lấy mẫu trace đã khai thác độ hiếm, đường đi ít gặp, trace bất thường hoặc cơ chế chú ý (attention) để ưu tiên trace nhiều thông tin hơn trace phổ biến [13, 14, 16].

Một cách đo tương đồng giữa hai trace là dựa trên tập dịch vụ mà chúng đi qua. Giả sử trace a đi qua tập dịch vụ S_a và trace b đi qua tập dịch vụ S_b . Độ tương đồng Jaccard [24, 25] được định nghĩa như trong Công thức (2.9).

$$J(S_a, S_b) = \frac{|S_a \cap S_b|}{|S_a \cup S_b|}. \quad (2.9)$$

Công thức (2.9) cho biết hai trace càng đi qua nhiều dịch vụ giống nhau thì độ tương đồng giữa chúng càng cao. Từ đó, khoảng cách Jaccard có thể được viết như trong công thức (2.10).

$$D(S_a, S_b) = 1 - J(S_a, S_b). \quad (2.10)$$

Trong việc lấy mẫu, độ tương đồng giúp hạn chế việc chọn nhiều trace gần như giống nhau. Tuy nhiên, sự đa dạng không nên được tối ưu độc lập. Một trace rất khác biệt nhưng không liên quan đến sự cố có thể không hữu ích cho RCA. Vì vậy, độ đa dạng của trace không nên được tối ưu độc lập, mà cần được kết hợp với tín hiệu bất thường và mức độ liên quan của trace đối với quá trình RCA.

2.5.3 Xây dựng hàm đánh giá trace

Hàm đánh giá *trace* là thành phần trung tâm của nhiều chiến lược lấy mẫu. Các nghiên cứu gần đây thường không đánh giá *trace* dựa trên một tín hiệu đơn lẻ, mà quyết định giữ hay bỏ thông qua việc tổng hợp nhiều nhóm tín hiệu khác nhau [14, 18, 23]. Các nhóm tín hiệu tiêu biểu thường được xem xét bao gồm:

- Mức độ bất thường và trạng thái lỗi của *trace*.
- Mức độ liên quan đến mục tiêu phân tích nguyên nhân gốc (RCA).
- Độ đa dạng và độ hiếm của cấu trúc *trace* trong hệ thống.
- Các *metric* trạng thái hệ thống theo thời gian thực.

Để tổng hợp các yếu tố này, các phương pháp thường sử dụng một hàm tính điểm tổng quát, trong đó mỗi nhóm tín hiệu sẽ đóng góp vào tổng điểm thông qua một trọng số nhất định. Đặc biệt, trong bối cảnh chấm điểm thích nghi (*adaptive scoring*), các trọng số này không bị gán cứng mà có thể thay đổi linh hoạt tùy thuộc vào mức ngân sách cho phép, trạng thái hiện tại của hệ thống, hoặc đặc điểm của khung thời gian xảy ra sự cố (*incident window*).

2.5.4 Kỹ thuật tổ hợp (ensemble) và đồng thuận (consensus)

Ở mức ngân sách thấp, quyết định lấy mẫu có thể có phương sai cao. Kỹ thuật tổ hợp (ensemble) là cách kết hợp nhiều mô hình, nhiều cấu hình hoặc nhiều lần chạy khác nhau để cải thiện độ ổn định của quyết định cuối cùng [26]. Đồng thuận (consensus) là cơ chế tổng hợp kết quả từ các thành viên trong tổ hợp bằng cách đếm số lần một trace được lựa chọn. Nếu $v(t)$ là số thành viên chọn trace t và K là tổng số thành viên trong tổ hợp, tỷ lệ đồng thuận được định nghĩa như trong Công thức (2.11).

$$\text{consensus}(t) = \frac{v(t)}{K}. \quad (2.11)$$

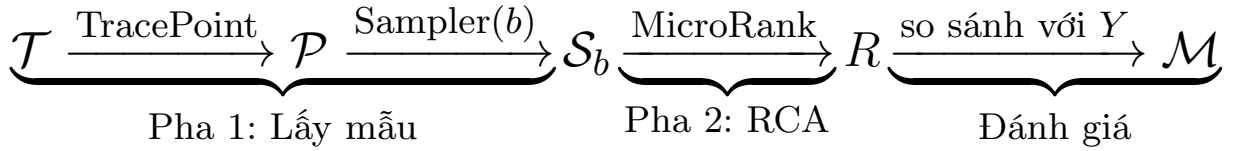
Cơ chế đồng thuận giúp giảm ảnh hưởng của các quyết định ngẫu nhiên, đặc biệt ở mức ngân sách rất nhỏ. Tuy nhiên, nếu ngưỡng đồng thuận quá chặt, tập trace được chọn có thể mất đi độ đa dạng và bỏ lỡ các trace hiếm nhưng hữu ích. Vì vậy, thiết kế tổ hợp cần cân bằng giữa độ ổn định và độ đa dạng của tập trace được giữ lại.

Chương 3

Phương pháp lấy mẫu trace thích nghi nhận biết RCA bằng đồng thuận đối sánh

Chương này trình bày phương pháp đề xuất cho bài toán lấy mẫu trace trong hệ thống vi dịch vụ nhằm phục vụ phân tích nguyên nhân gốc. Trong các hệ thống vi dịch vụ, số lượng trace được sinh ra trong quá trình vận hành thường rất lớn, trong khi việc lưu trữ và xử lý toàn bộ trace gây ra chi phí đáng kể. Do đó, lấy mẫu trace là một bước cần thiết để giảm tải dữ liệu. Tuy nhiên, trong bối cảnh phân tích nguyên nhân gốc, mục tiêu của lấy mẫu không chỉ là giảm số lượng trace, mà còn phải bảo toàn những tín hiệu quan trọng giúp hệ thống RCA xác định đúng dịch vụ gây lỗi.

Quy trình lấy mẫu và đánh giá trong khóa luận được tổ chức thành hai pha chính, như minh họa trong Hình 3.1.



Hình 3.1: Pipeline hai pha của phương pháp đề xuất.

Trong đó, $\mathcal{T}$ là tập trace gốc. Bộ dụng TracePoint chuyển mỗi trace thành biểu diễn rút gọn, tạo thành tập $\mathcal{P}$. Trên tập biểu diễn này, bộ lấy mẫu chọn ra tập trace con $\mathcal{S}_b$ thỏa mãn ngân sách lấy mẫu b .

Sau pha lấy mẫu, tập trace được giữ lại được đưa vào bộ RCA là **MicroRank** để sinh ra danh sách dịch vụ được xếp hạng theo mức độ nghi ngờ là nguyên nhân gốc. Cuối cùng, danh sách xếp hạng này được so sánh với nhãn nguyên nhân gốc thực tế (*ground truth*) để tính các chỉ số đánh giá như **A@1**, **A@3** và **MRR**.

3.1 Tổng quan phương pháp và kiến trúc hệ thống

3.1.1 Định hướng và cách tiếp cận

Dựa trên những gì đã phân tích, phương pháp trong khóa luận này không chỉ xem bộ lấy mẫu trace như một bộ lọc dữ liệu độc lập, mà là một thành phần nằm trong một quy trình xử lý (pipeline) phân tích nguyên nhân gốc. Nói cách khác, bộ lấy mẫu không chỉ có nhiệm vụ giảm số lượng trace được lưu trữ, mà còn phải bảo toàn những tín hiệu cần thiết để bộ RCA phía sau có thể xếp hạng đúng dịch vụ gây lỗi. Hơn nữa, việc phát triển phương pháp phải đặt trong khuôn khổ yêu cầu cho sự đơn giản, dễ sử dụng, nhẹ nhàng nhưng vẫn đạt được kết quả tốt. Trong khóa luận này, bộ RCA được sử dụng là **MicroRank**; do đó, thiết kế bộ lấy mẫu cần được đặt trong mối quan hệ trực tiếp với cơ chế hoạt động của MicroRank.

Từ định hướng đó, khóa luận tiếp cận bài toán theo hướng lấy mẫu nhận biết RCA (*RCA-aware sampling*). Thay vì tối ưu từng trace riêng lẻ theo mức độ bất thường, phương pháp đề xuất tối ưu thành phần của tập trace được giữ lại. Tập trace này cần cân bằng giữa trace lỗi và trace bình thường, giữa trace có liên quan đến dịch vụ nghi ngờ và trace đóng vai trò nền đối sánh, đồng thời vẫn phải tuân thủ ràng buộc ngân sách lấy mẫu rất nhỏ.

Về mặt phát triển phương pháp, khóa luận đi theo lộ trình tăng dần mức độ thông tin được sử dụng. Trước hết, phương pháp nền chỉ khai thác các tín hiệu có sẵn ở mức trace như độ trễ, số lượng span, cờ lỗi, thời điểm xuất hiện và đường đi dịch vụ. Sau đó, phương pháp được mở rộng theo hướng nhận biết RCA rõ hơn bằng cách đưa vào tư duy đối sánh giữa trace bất thường và trace bình thường. Cuối cùng, khi có thêm metric hệ thống, phương pháp tiếp tục được mở rộng bằng metric gốc, cơ chế đối sánh có dấu, tổ hợp nhiều lần chạy và đồng thuận để tăng độ ổn định của quyết định lấy mẫu trong điều kiện ngân sách cực thấp.

Từ lộ trình trên, khóa luận đề xuất phương pháp tổng thể **ARCS** – *Adaptive RCA-aware Consensus Sampling*, tạm dịch là **lấy mẫu trace thích nghi nhận biết RCA bằng đồng thuận dựa trên đối sánh**. ARCS được cụ thể hóa qua hai nhánh phương pháp chính.

Nhánh thứ nhất là **lấy mẫu trace thích nghi dựa trên chấm điểm và đa dạng** (*Trace-Adaptive Scoring with Diversity* – **TASD**). TASD đóng vai trò phương pháp

nền, tập trung vào việc khai thác các tín hiệu có sẵn ở mức trace và kiểm soát độ đa dạng của tập trace được giữ lại.

Nhánh thứ hai là **lấy mẫu đối sánh có dấu dựa trên metric gốc và tổ hợp đồng thuận** (*Metric-native Ensemble Signed-Contrast Sampling – MESC*). MESC là phần mở rộng của T ASD, trong đó phương pháp bổ sung metric gốc của hệ thống, cơ chế đối sánh có dấu, tổ hợp nhiều lần chạy và đồng thuận để tăng độ ổn định của quyết định lấy mẫu.

3.1.2 Pipeline hai pha

Từ pipeline tổng thể trong Hình 3.1, có thể thấy phương pháp đề xuất tách quá trình thực nghiệm thành hai nhiệm vụ độc lập: lấy mẫu trace và phân tích nguyên nhân gốc. Cách tách này giúp đánh giá sampler không chỉ theo khả năng giảm dữ liệu, mà còn theo mức độ bảo toàn tín hiệu cần thiết cho RCA.

Trong pha lấy mẫu, sampler làm việc trên biểu diễn rút gọn của trace để tính điểm và lựa chọn tập trace con theo ngân sách. Pha này quyết định những trace nào được giữ lại để tiếp tục tham gia vào quá trình phân tích nguyên nhân gốc.

Trong pha RCA, **MicroRank** được chạy trực tiếp trên tập trace sau lấy mẫu. Do đó, kết quả đánh giá phản ánh tác động thực tế của sampling đến cấu trúc đồ thị dịch vụ, phân vùng trace bình thường/bất thường và thứ hạng nguyên nhân gốc cuối cùng.

3.1.3 Biểu diễn trace đầu vào

Để đảm bảo sampler nhẹ và có thể xử lý theo dòng, mỗi trace được rút gọn thành một biểu diễn mức trace. Cụ thể, một trace t được biểu diễn như trong Công thức 3.1:

$$t = (id, ts, d, c, e, P), \quad (3.1)$$

trong đó:

- id : mã định danh trace;
- ts : thời điểm bắt đầu trace;
- d : độ trễ của trace, tương ứng với `duration_ms`;

- c : số span trong trace, tương ứng với `span_count`;
- e : cờ lỗi, tương ứng với `has_error`;
- P : tập hoặc chuỗi dịch vụ mà trace đi qua.

Biểu diễn này có hai ưu điểm. Thứ nhất, nó đủ đơn giản để áp dụng trong môi trường xử lý theo dòng. Thứ hai, nó tương thích với dữ liệu đầu vào của benchmark và các script hiện có.

Tuy nhiên, biểu diễn này cũng có hạn chế: sampler không trực tiếp nhìn thấy toàn bộ đồ thị span chi tiết. Vì vậy, các tín hiệu như causal graph (đồ thị nhân quả), dependency graph (đồ thị phụ thuộc) hay motif cấu trúc sâu chỉ có thể được xấp xỉ từ service path.

3.1.4 Ràng buộc xử lý theo dòng

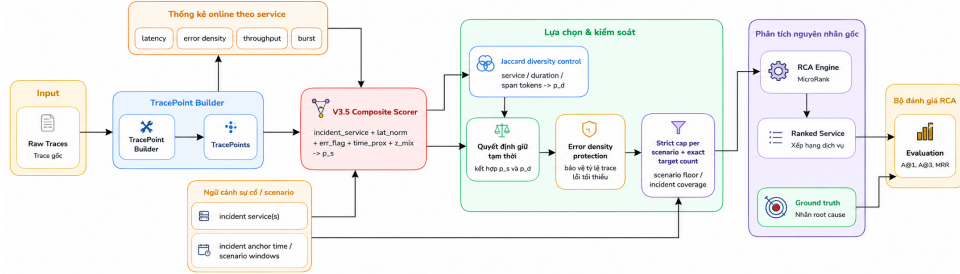
Sampler được thiết kế theo nguyên tắc online-safe (an toàn theo dòng). Tại thời điểm quyết định cho trace t_i , sampler chỉ được dùng:

- thông tin của chính trace t_i ;
- thống kê lịch sử từ các trace đã quan sát trước đó;
- metric hệ thống tại hoặc trước thời điểm tương ứng;
- cấu hình ngân sách và các ràng buộc đã biết trước.

Sampler không được dùng thông tin tương lai, ví dụ không được tính phân phối điểm trên toàn bộ dataset rồi quay lại quyết định cho trace đầu. Ràng buộc này quan trọng vì mục tiêu của phương pháp là tiến gần tới bối cảnh tail sampling trong hệ thống quan sát phân tán. Trong OpenTelemetry, tail sampling processor đưa ra quyết định sau khi gom các span thuộc cùng trace và dựa trên các policy đã định nghĩa; đồng thời, toàn bộ span của một trace cần đi tới cùng collector instance để quyết định sampling hiệu quả [15].

3.2 Phương pháp nền TASD: lấy mẫu trace thích nghi dựa trên chấm điểm và đa dạng

3.2.1 Mục tiêu thiết kế



Hình 3.2: Quy trình lấy mẫu TASD dựa trên tập trace gốc.

TASD là phương pháp cốt lõi được đề xuất trong khóa luận. Mục tiêu của phương pháp là thiết kế một bộ lấy mẫu toàn diện, có khả năng đáp ứng các tiêu chí khắt khe sau:

- **Tự chủ dữ liệu:** Chỉ sử dụng thông tin ở mức *trace* và các chỉ số thống kê suy xuất từ chính tập *trace* đó.
- **Không phụ thuộc huấn luyện:** Khả năng hoạt động linh hoạt mà không yêu cầu quá trình huấn luyện có giám sát (*unsupervised*).
- **Xử lý theo dòng (*streaming*):** Khả năng đưa ra quyết định trực tuyến (*online*) trên luồng dữ liệu đến liên tục.
- **Tuân thủ ngân sách nghiêm ngặt:** Đảm bảo tổng dung lượng dữ liệu giữ lại không vượt quá giới hạn tài nguyên cho phép.
- **Kiểm soát độ đa dạng:** Ngăn chặn việc lưu trữ các *trace* có cấu trúc trùng lặp, qua đó tối ưu hóa không gian mẫu.
- **Bảo đảm chất lượng RCA:** Giữ lại đầy đủ các *trace* mang thông tin bất thường và bối cảnh sự cố, cung cấp đầu vào chất lượng cao để thực thi thuật toán *MicroRank*.

Ở giai đoạn này, T ASD chưa cố gắng tái hiện đầy đủ TraStrainer. TraStrainer sử dụng system runtime state và trace diversity trong một online sampler [14]. T ASD chỉ khai thác một phần ý tưởng tương tự nhưng đặt trong ràng buộc dữ liệu của khóa luận: thông tin chính là duration, span count, error flag, timestamp và dịch vụ path. Quy trình lấy mẫu được mô tả trong Hình 3.2.

3.2.2 Các đặc trưng trace và tín hiệu chấm điểm

T ASD gán cho mỗi trace một điểm thể hiện độ quan trọng đối với sampling. Các tín hiệu chính gồm:

Bảng 3.1: Các nhóm tín hiệu trong T ASD.

Nhóm tín hiệu	Tên tiếng Anh	Ý nghĩa
Tín hiệu lỗi	error signal	Trace có chứa lỗi hay không.
Tín hiệu độ trễ	latency signal	Trace có thời gian xử lý cao bất thường hay không.
Tín hiệu gần sự cố	incident proximity	Trace có nằm gần cửa sổ thời gian sự cố hay không.
Tín hiệu dịch vụ liên quan	incident service signal	Trace có đi qua dịch vụ liên quan đến sự cố hay không.
Tín hiệu độ phức tạp	trace complexity signal	Trace có nhiều span hoặc đi qua đường dịch vụ phức tạp hay không.

Gọi các hàm đặc trưng tương ứng với các chỉ số trong Bảng 3.1 là $f_e(t)$, $f_l(t)$, $f_i(t)$, $f_p(t)$ và $f_c(t)$. Điểm mức trace được mô tả khái quát:

$$S_{trace}(t) = w_e f_e(t) + w_l f_l(t) + w_i f_i(t) + w_p f_p(t) + w_c f_c(t). \quad (3.2)$$

Trong công thức (3.2) w_e, w_l, w_i, w_p, w_c là các trọng số điều chỉnh tầm quan trọng của từng tín hiệu. Thiết kế này có tính giải thích cao vì mỗi thành phần điểm đều có thể truy vết về một giả thuyết cụ thể: trace lỗi đáng chú ý hơn trace không lỗi; trace gần cửa sổ sự cố đáng chú ý hơn trace xa sự cố; trace đi qua dịch vụ liên quan đáng chú ý hơn trace không liên quan.

3.2.3 Tín hiệu trạng thái hệ thống suy ra từ trace

Do TASD không giả định có đầy đủ metric hệ thống thật, phương pháp sử dụng các thống kê online suy ra từ trace để xấp xỉ trạng thái hệ thống. Các thống kê này được duy trì theo dịch vụ hoặc theo cửa sổ thời gian:

- độ trễ trung bình trượt;
- tỷ lệ lỗi trượt;
- mật độ lỗi trong cửa sổ gần;
- mức tăng đột biến về số lượng trace;
- độ lệch robust của latency so với lịch sử.

Với một dịch vụ s , giả sử $L_s(t)$ là latency quan sát được tại thời điểm t . Một cách cập nhật đơn giản bằng trung bình động hàm mũ (EMA) sau khi áp dụng công thức (2.8) được mô tả như công thức (3.3) sau:

$$EMA_s(t) = \alpha L_s(t) + (1 - \alpha) EMA_s(t - 1), \quad (3.3)$$

trong đó $\alpha \in (0, 1]$ là hệ số làm mượt.

Để phát hiện bất thường ổn định hơn trước các giá trị ngoại lai, có thể sử dụng MAD hoặc các thước đo độ phân tán bền vững liên quan. Tương tự, áp dụng công thức (2.7) điểm bất thường ổn định có thể được mô tả:

$$z_{robust}(t) = \frac{L_s(t) - median(L_s)}{MAD(L_s) + \epsilon}, \quad (3.4)$$

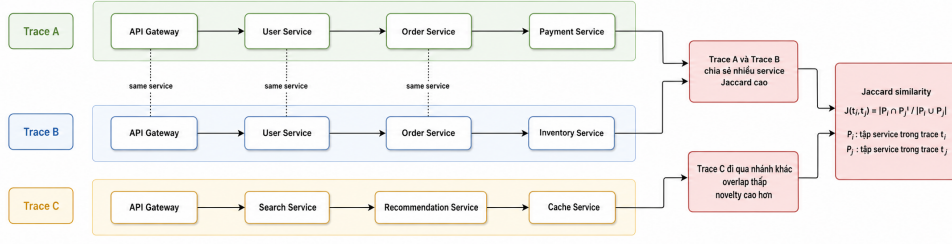
trong công thức (3.4) ϵ là hằng số nhỏ để tránh chia cho 0.

Từ các thống kê này, TASD tạo ra điểm trạng thái hệ thống như trong Công thức (3.5).

$$S_{state}(t) = a_l g_l(t) + a_e g_e(t) + a_b g_b(t), \quad (3.5)$$

trong đó $g_l(t)$ là tín hiệu latency, $g_e(t)$ là tín hiệu error density và $g_b(t)$ là tín hiệu burst.

3.2.4 Đa dạng hóa trace bằng độ tương đồng Jaccard



Hình 3.3: Đa dạng hóa trace bằng độ tương đồng Jaccard giữa các chuỗi dịch vụ.

Nếu bộ lấy mẫu chỉ ưu tiên chọn các *trace* có điểm bất thường cao nhất, tập dữ liệu được giữ lại có thể chứa rất nhiều *trace* có cấu trúc gần như giống nhau. Điều này làm giảm độ phủ của đồ thị dịch vụ và gây lãng phí ngân sách cho các mẫu hành vi lặp lại. Để khắc phục hạn chế này, phương pháp TASD tích hợp thêm thành phần đa dạng hóa *trace* (*trace diversity*).

Thay vì tính toán lại từ đầu, khóa luận sử dụng trực tiếp khoảng cách Jaccard dựa trên tập dịch vụ mà *trace* đi qua, như đã định nghĩa tại Công thức (2.10). Dựa trên cơ sở đó, điểm đa dạng của một *trace* t mới đến được xác định thông qua việc so sánh nó với tập các *trace* đã được lưu trữ, thể hiện qua công thức (3.6):

$$S_{diversity}(t) = 1 - \max_{u \in \mathcal{S}_{current}} J(t, u), \quad (3.6)$$

trong đó $\mathcal{S}_{current}$ là tập hợp các *trace* đang được giữ trong bộ đệm tại thời điểm đánh giá t , và $J(t, u)$ là độ tương đồng Jaccard giữa *trace* t và *trace* u .

Thông qua công thức này, nếu *trace* t mang một đường đi hoàn toàn khác biệt so với các *trace* hiện có trong $\mathcal{S}_{current}$, giá trị tương đồng lớn nhất $\max J(t, u)$ sẽ nhỏ, dẫn đến điểm đa dạng $S_{diversity}(t)$ cao tức thể hiện tính mới mẻ (*novelty*). Ngược lại, nếu t đi qua các dịch vụ gần như trùng khớp với một *trace* đã được lưu trước đó, điểm đa dạng sẽ bị phạt và hạ xuống mức thấp.

Cơ chế này được minh họa trực quan trong Hình 3.3, đảm bảo hệ thống chủ động hạn chế các *trace* có mức độ chồng lặp dịch vụ cao, từ đó nhường không gian lưu trữ quý giá cho các *trace* mang lại thông tin cấu trúc khác biệt hơn.

3.2.5 Hàm chấm điểm tổng hợp và quy tắc chọn trace

Điểm tổng hợp của TASD được định nghĩa như trong Công thức (3.7).

$$S_{\text{TASD}}(t) = \lambda_1 S_{\text{trace}}(t) + \lambda_2 S_{\text{state}}(t) + \lambda_3 S_{\text{diversity}}(t). \quad (3.7)$$

Trong đó:

- $S_{\text{trace}}(t)$ phản ánh mức quan trọng cục bộ của trace;
- $S_{\text{state}}(t)$ phản ánh trạng thái bất thường suy ra từ lịch sử trace;
- $S_{\text{diversity}}(t)$ phản ánh độ mới của đường đi dịch vụ;
- $\lambda_1, \lambda_2, \lambda_3$ là các trọng số điều chỉnh sự đánh đổi giữa các nhóm tín hiệu.

Dựa trên điểm tổng hợp này, quy tắc chọn trace ban đầu được mô tả như sau.

$$\text{keep}(t) = \begin{cases} 1, & S_{\text{TASD}}(t) \geq \theta_b, \\ 0, & \text{ngược lại.} \end{cases} \quad (3.8)$$

Trong Công thức (3.8), θ_b là ngưỡng phụ thuộc ngân sách. Sau bước chọn ban đầu, sampler áp dụng strict cap (chặn ngân sách chặt). Gọi $k_b = \lfloor b|\mathcal{T}| \rfloor$. Nếu số trace giữ vượt k_b , chỉ k_b trace có điểm cao nhất hoặc có vai trò bảo vệ ràng buộc được giữ lại:

$$|\mathcal{S}_b| \leq k_b. \quad (3.9)$$

Công thức (3.9) đảm bảo số trace được giữ lại không vượt quá ngân sách cho phép. Nếu số trace được chọn ban đầu vượt quá k_b , sampler chỉ giữ lại k_b trace có điểm cao nhất hoặc có vai trò bảo vệ ràng buộc.

Ngoài strict cap, TASD còn cần các floor, tức các ngưỡng tối thiểu, để tránh trường hợp tập trace sau lấy mẫu không đủ dữ liệu cho RCA:

- số trace tối thiểu trong cửa sổ sự cố;
- tỷ lệ trace lỗi tối thiểu;
- độ phủ dịch vụ tối thiểu;
- số trace tối thiểu theo scenario nếu benchmark gồm nhiều scenario.

3.2.6 Hạn chế của TASD

TASD là một phương pháp nền quan trọng vì nó nhẹ, dễ giải thích và không phụ thuộc vào metric thật. Tuy nhiên, phương pháp này có ba hạn chế chính.

Thứ nhất, TASD vẫn dựa nhiều vào tín hiệu suy ra từ trace. Những tín hiệu này không tương đương với metric hệ thống thật như CPU, memory, network I/O hay workload. Khi lỗi biểu hiện mạnh ở metric nhưng không biểu hiện rõ trong trace duration hoặc error flag, TASD có thể bỏ sót trace quan trọng.

Thứ hai, TASD có thể nhầm giữa trace bất thường và trace hữu ích cho RCA. Một trace lỗi không liên quan root cause có thể được chấm cao, trong khi một trace bình thường có giá trị đối sánh lại bị chấm thấp.

Cuối cùng, ở ngân sách cực nhỏ, phương sai quyết định rất cao. Nếu chỉ chạy một bộ lấy mẫu duy nhất, một trace có thể được giữ hoặc bị loại do nhiễu trong thứ hạng. Đây là động cơ dẫn tới các biến thể về đồng thuận và kết hợp ở các phần sau.

3.3 Mở rộng chấm điểm đối sánh cho MicroRank

3.3.1 Vì sao MicroRank cần đối sánh

MicroRank là phương pháp RCA dựa trên trace, trong đó thông tin từ trace bình thường và trace bất thường được dùng để xếp hạng dịch vụ nghi ngờ [3]. Về trực giác, một dịch vụ có khả năng là root cause nếu nó xuất hiện nhiều trong trace bất thường và ít xuất hiện trong trace bình thường. Ý tưởng này gần với nhóm kỹ thuật spectrum-based fault localization (định vị lỗi dựa trên phổ thực thi), trong đó một thực thể được xem là đáng nghi nếu nó liên quan nhiều tới ca lỗi và ít liên quan tới ca đúng [27].

Gọi một dịch vụ là s . Có thể mô tả bốn số đếm:

- ef_s : số trace bất thường chứa dịch vụ s ;
- ep_s : số trace bình thường chứa dịch vụ s ;
- nf_s : số trace bất thường không chứa dịch vụ s ;
- np_s : số trace bình thường không chứa dịch vụ s .

Một dịch vụ đáng nghi thường có ef_s cao và ep_s thấp. Vì vậy, nếu sampler giữ quá nhiều trace lỗi không liên quan root cause, nf_s hoặc các số đếm nhiều khác có thể tăng, khiến điểm nghi ngờ của root cause thật bị giảm tương đối. Ngược lại, nếu sampler loại bỏ quá nhiều trace bình thường, MicroRank mất nền đối sánh và ranking trở nên kém ổn định.

Từ góc nhìn này, mục tiêu của sampler không phải là chọn top- k trace xấu nhất. Mà là chọn một hỗn hợp trace sao cho khi đưa vào MicroRank, dịch vụ là nguyên nhân gốc của lỗi trở nên nổi bật nhất.

3.3.2 Mô hình bốn vùng RCA

Để đưa tư duy RCA vào sampler, phương pháp mở rộng TASD bằng mô hình bốn vùng RCA. Mỗi trace được xét theo hai trục:

- trace có bất thường/lỗi hay không;
- trace có đi qua dịch vụ nghi ngờ hay không.

Từ đó hình thành bốn vùng trong Bảng 3.2.

Bảng 3.2: Mô hình bốn vùng RCA cho trace sampling.

Vùng	Điều kiện	Ý nghĩa đối với RCA
Vàng (Gold)	Trace lỗi/bất thường và đi qua dịch vụ nghi ngờ	Tín hiệu mạnh, có khả năng làm root cause nổi bật.
Bạc (Silver)	Trace bình thường và không đi qua dịch vụ nghi ngờ	Nền đối sánh sạch, giúp phân biệt hành vi bình thường.
Nhiều (Noise)	Trace lỗi/bất thường nhưng không đi qua dịch vụ nghi ngờ	Có thể làm loãng tín hiệu root cause nếu giữ quá nhiều.
Pha trộn (Mixed)	Trace bình thường nhưng đi qua dịch vụ nghi ngờ	Có thể hữu ích để phân biệt lỗi thật với dịch vụ xuất hiện phổ biến.

Điểm quan trọng của mô hình này là trace lỗi không còn được coi là luôn tốt. Trace lỗi chỉ thực sự có giá trị cao nếu nó hỗ trợ việc làm nổi bật dịch vụ nghi

ngờ. Tương tự, trace bình thường không còn bị xem là vô ích; một số trace bình thường cần được giữ để tạo nền so sánh.

3.3.3 Chấm điểm đối sánh RCA

Gọi $S_{rca}(t)$ là điểm đối sánh RCA của trace t . Điểm này được xác định dựa trên vùng RCA mà trace thuộc về. Một dạng điểm minh họa:

$$S_{rca}(t) = \begin{cases} q_{gold}, & t \in Gold, \\ q_{silver}, & t \in Silver, \\ q_{mixed}, & t \in Mixed, \\ q_{noise}, & t \in Noise. \end{cases} \quad (3.10)$$

Trong Công thức (3.10), các hệ số biểu diễn mức ưu tiên của các vùng RCA tương ứng. Để nhấn mạnh vai trò của các trace có giá trị đối sánh cao, các hệ số này cần thỏa mãn ràng buộc ưu tiên trong Công thức (3.11).

$$q_{gold} > q_{mixed}, \quad q_{gold} > q_{noise}. \quad (3.11)$$

Tuy nhiên, q_{noise} không nên bằng 0. Một số trace lỗi ban đầu có thể chưa đi qua dịch vụ được nhận diện là nghi ngờ do lịch sử chưa đủ. Nếu phạt độ nhiễu trace quá mạnh, bộ lấy mẫu có thể bỏ mất trace lỗi sớm quan trọng. Đây là lý do các biến thể như T ASD-RCA làm mềm điểm quadrant và giảm nguy cơ phạt quá mạnh trace lỗi sớm. Điểm tổng hợp nhận biết RCA (RCA-aware) được viết:

$$S_{RCA}(t) = \alpha S_{TASD}(t) + \delta S_{rca}(t). \quad (3.12)$$

Trong (3.12) δ điều chỉnh mức ảnh hưởng của tín hiệu RCA-aware. Khi $\delta = 0$, phương pháp quay về T ASD thuần trace-only. Khi δ tăng, sampler ưu tiên nhiều hơn các trace có giá trị đối sánh cho MicroRank.

3.3.4 Điều tiết tỷ lệ trace lỗi và trace bình thường

Một bộ lấy mẫu nhận biết RCA cần tránh hai cực đoan. Cực đoan thứ nhất là giữ quá nhiều trace lỗi. Điều này có vẻ hợp lý với anomaly detection, nhưng có thể làm MicroRank thiếu nền bình thường để đối sánh. Cực đoan thứ hai là giữ quá nhiều trace bình thường, khiến tín hiệu sự cố không đủ mạnh [3].

Do đó, phương pháp duy trì các ràng buộc tỷ lệ giữa trace lỗi và trace bình thường trong tập trace được giữ lại. Cụ thể, bộ lấy mẫu cần đảm bảo tỷ lệ trace lỗi không quá thấp để vẫn giữ được tín hiệu sự cố, nhưng cũng không quá cao để tránh làm mất dữ liệu nền. Đồng thời, tập trace sau lấy mẫu cần giữ lại một lượng trace bình thường tối thiểu nhằm phục vụ bước đối sánh của MicroRank.

Các ngưỡng tỷ lệ này có thể phụ thuộc vào ngân sách lấy mẫu. Ở mức 0.1%, sampler cần ưu tiên lỗi tín hiệu mạnh nhưng vẫn phải giữ một số trace bình thường cần thiết. Ở các mức 1.0% và 2.5%, phần ngân sách tăng thêm nên được dùng để mở rộng ngữ cảnh, tăng độ đa dạng và củng cố khả năng đối sánh.

3.4 Phương pháp MESC: lấy mẫu đối sánh có dấu dựa trên metric gốc và đồng thuận tổ hợp

3.4.1 Động cơ đưa metric gốc vào bộ lấy mẫu

TASD và Lấy mẫu dựa trên trace theo hướng nhận biết RCA (RCA-aware trace-only sampling) khai thác thông tin từ trace. Tuy nhiên, nhiều sự cố trong vi dịch vụ biểu hiện rõ ở metric hệ thống hơn là ở trace đơn lẻ. Ví dụ, một dịch vụ có thể bị quá tải CPU, tăng bộ nhớ, giảm tỷ lệ thành công hoặc tăng tải trước khi mọi trace đi qua nó đều có lỗi.

TraStrainer đã chỉ ra rằng chỉ dựa trên độ đa dạng của trace là chưa đủ; trạng thái hệ thống trong thời gian thực là một yếu tố quan trọng, đặc biệt trong các trường hợp lỗi hệ thống [14]. Vì vậy, phương pháp MESC đưa thêm metric gốc (metric-native signal – tín hiệu metric gốc) vào quá trình chấm điểm.

Các loại dữ liệu metric được xét trong thiết kế gồm:

- mức sử dụng CPU (*CPU usage*);
- mức sử dụng bộ nhớ (*memory usage*);
- độ trễ (*latency*);
- tải/lưu lượng xử lý (*workload*);
- tỷ lệ thành công (*success rate*);
- các tín hiệu metric tương đương khác.

Mục tiêu không phải là thay thế trace bằng metric, mà là kết hợp metric với trace để tạo bằng chứng mạnh hơn về dịch vụ nghi ngờ.

3.4.2 Ánh xạ trace với metric theo dịch vụ và thời gian

Để tổng hợp tín hiệu từ hệ thống, mỗi *trace* t cần được đối chiếu với các chỉ số hạ tầng tương ứng. Với mỗi *trace* t , bộ lấy mẫu xác định được thời điểm bắt đầu $ts(t)$ và tập dịch vụ $P(t)$ mà nó đi qua. Với mỗi dịch vụ $s \in P(t)$, hệ thống sẽ trích xuất vector *metric* $m(s, \tau)$ tại thời điểm τ nằm trong một cửa sổ thời gian sát với lúc *trace* phát sinh.

Tập hợp các *metric* liên quan trực tiếp đến *trace* t được định nghĩa theo công thức (3.13):

$$M(t) = \{m(s, \tau) \mid s \in P(t), \tau \in [ts(t) - \Delta t, ts(t) + \Delta t]\}, \quad (3.13)$$

trong đó Δt là sai số thời gian cho phép để đảm bảo dữ liệu *metric* và *trace* được đồng bộ.

Từ tập dữ liệu $M(t)$ này, phương pháp TASD sẽ phân tích và trích xuất ra bốn đại lượng đặc trưng nhằm phục vụ việc chấm điểm, bao gồm:

- **Bảng chứng metric (E_m):** Các tín hiệu gốc từ *metric* đóng vai trò ủng hộ hoặc phản bác giả thuyết *trace* đang gặp sự cố.
- **Cường độ bất thường (I_m):** Mức độ sai lệch của *metric* so với trạng thái nền (được đo lường thông qua các giá trị như *z-score* hoặc khoảng cách MAD).
- **Độ phủ dữ liệu (C_m):** Tỷ lệ các dịch vụ trong $P(t)$ có thu thập được *metric* hợp lệ, đảm bảo tính toàn vẹn của dữ liệu đối sánh.
- **Độ tin cậy ánh xạ (R_m):** Trọng số đánh giá chất lượng đồng bộ thời gian giữa *trace* và các *time bucket* của *metric*.

Dựa trên các đại lượng đặc trưng này, điểm *metric* tổng hợp của *trace* t được xác định thông qua một hàm mục tiêu h , thể hiện qua công thức (3.14):

$$S_{metric}(t) = h(E_m, I_m, C_m, R_m). \quad (3.14)$$

3.4.3 Đối sánh có dấu

Một hạn chế của điểm metric thông thường là nó thường chỉ đo “mức bất thường”. Nhưng đối với RCA, không phải mọi bất thường đều ủng hộ quyết định giữ trace. Một metric bất thường ở dịch vụ không liên quan root cause có thể là nhiều; ngược lại, một metric bình thường ở dịch vụ nghi ngờ có thể là tín hiệu phản bác giả thuyết root cause.

Vì vậy, MESC dùng **đối sánh có dấu (signed contrast)**. Ý tưởng là tách tín hiệu metric thành hai phần:

- $C^+(t)$: tín hiệu ủng hộ trace hoặc dịch vụ nghi ngờ;
- $C^-(t)$: tín hiệu phản bác hoặc có nguy cơ gây nhiễu.

Điểm đối sánh có dấu $C(t)$ là tổng hai thành phần, ví dụ: $C(t) = C^+(t) - C^-(t)$ (trong trường hợp $C^-(t)$ là giá trị tuyệt đối của tín hiệu phản bác)., nếu $C(t)$ cao, trace có bằng chứng metric phù hợp với cấu trúc đối sánh RCA. Nếu $C(t)$ thấp hoặc âm, trace có thể là nhiễu hoặc không giúp MicroRank phân biệt root cause.

Theo đó, điểm MESC được chấm như công thức (3.15):

$$S_{MESC}(t) = \alpha S_{trace}(t) + \beta S_{rca}(t) + \lambda C(t) + \rho S_{diversity}(t). \quad (3.15)$$

Trong đó:

- $S_{trace}(t)$: giữ lại khả năng giải thích từ TASD;
- $S_{rca}(t)$: đưa vào mô hình bốn vùng RCA;
- $C(t)$: điểm đối sánh có dấu khai thác từ các bằng chứng *metric* gốc.
- $S_{diversity}(t)$: giữ vai trò chống trùng lặp dịch vụ path.

3.4.4 Tổ hợp nhiều thành viên

Ở ngân sách cực nhỏ, quyết định sampling có phương sai cao. Một trace nằm sát ngưỡng có thể được giữ ở lần chạy này nhưng bị loại ở lần chạy khác. Để giảm phương sai, MESC dùng tổ hợp (ensemble) nhiều thành viên. Ý tưởng tổ hợp trong

học máy đã được nghiên cứu rộng rãi, trong đó nhiều mô hình hoặc nhiều lần chạy được kết hợp để tạo quyết định ổn định hơn [26].

Gọi K là số thành viên trong ensemble. Mỗi thành viên sinh ra một tập trace:

$$\mathcal{S}^{(1)}, \mathcal{S}^{(2)}, \dots, \mathcal{S}^{(K)}. \quad (3.16)$$

Các thành viên trong (3.16) có thể khác nhau bởi số hạt giống, nhiễu nhỏ trong việc chấm điểm hoặc cấu hình phụ. Khi đó, thay vì tin vào một kết quả duy nhất, phương pháp xét mức ổn định của trace qua nhiều lần chạy.

3.4.5 Cơ chế đồng thuận

Với một trace t , số phiếu đồng thuận được định nghĩa:

$$v(t) = \sum_{k=1}^K \mathbb{I}(t \in \mathcal{S}^{(k)}), \quad (3.17)$$

trong đó $\mathbb{I}$ là hàm chỉ báo.

Tỷ lệ đồng thuận:

$$\text{consensus}(t) = \frac{v(t)}{K}. \quad (3.18)$$

Trace được ưu tiên nếu:

$$\text{consensus}(t) \geq \theta_c, \quad (3.19)$$

trong đó θ_c là ngưỡng đồng thuận.

Cơ chế đồng thuận có hai tác dụng. Thứ nhất, nó loại bớt trace chỉ được chọn ngẫu nhiên bởi một seed, dựa theo công thức xác định trace ưu tiên thông qua (3.18) và (3.19). Thứ hai, nó chuyển đơn vị tối ưu từ chấm điểm từng trace riêng lẻ (trace-level scoring) sang độ ổn định của tập được chọn (set-level stability).

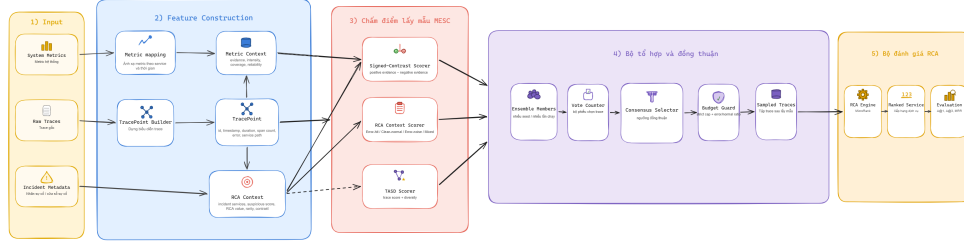
Khi nhiều trace có cùng số phiếu đồng thuận được tính theo công thức (3.17), phương pháp sử dụng một số tiêu chí phụ để xác định thứ tự ưu tiên. Các tiêu chí này bao gồm điểm đối sánh RCA, mức độ đối sánh có dấu giữa trace bất thường và trace bình thường, cờ lỗi, mức độ đa dạng của đường đi dịch vụ và độ tin cậy khi ánh xạ trace với metric. Các tiêu chí phụ này không thay thế cơ chế đồng thuận, mà chỉ được dùng để đánh giá trong trường hợp nhiều trace có cùng số phiếu.

Một điểm tổng hợp cho ranking cuối có thể viết theo công thức (3.20):

$$S_{final}(t) = \eta_1 consensus(t) + \eta_2 \tilde{S}_{MESC}(t) + \eta_3 S_{diversity}(t) + \eta_4 R_{metric}(t), \quad (3.20)$$

trong đó $\tilde{S}_{MESC}(t)$ là điểm MESC đã chuẩn hóa và $R_{metric}(t)$ là độ tin cậy metric.

3.4.6 Quy trình chọn trace cuối cùng



Hình 3.4: Quy trình lấy mẫu MESC kết hợp metric gốc.

Quy trình MESC được minh họa thông qua Hình 3.4 gồm các bước:

1. Dựng biểu diễn trace $t = (id, ts, d, c, e, P)$
2. Đọc và chuẩn hóa metric gốc theo dịch vụ và theo thời gian.
3. Ánh xạ trace với metric để tạo ngữ cảnh metric tương ứng với từng trace.
4. Tính các điểm thành phần, bao gồm điểm dựa trên tín hiệu trace, điểm nhận biết RCA và điểm đối sánh có dấu.
5. Chạy nhiều thành viên tổ hợp với các hạt giống ngẫu nhiên khác nhau để tạo các tập trace ứng viên $\mathcal{S}^{(k)}$.
6. Tính số phiếu chọn $v(t)$ và tỷ lệ đồng thuận $consensus(t)$ cho từng trace.
7. xếp hạng trace theo số phiếu đồng thuận và tổ hợp.
8. Áp dụng giới hạn ngân sách nghiêm ngặt và các ràng buộc về tỷ lệ giữa trace lỗi và trace bình thường.
9. Sinh tập trace cuối cùng $\mathcal{S}_b$ để đưa vào MicroRank.

3.4.7 So sánh TASD và MESC

TASD và MESC là hai nhánh phương pháp chính trong ARCS, nhưng chúng phục vụ hai mức thiết kế khác nhau. TASD đóng vai trò phương pháp nền, tập trung vào việc khai thác các tín hiệu có sẵn ở mức trace và đảm bảo bộ lấy mẫu vẫn nhẹ, dễ giải thích, có thể xử lý theo dòng. Ngược lại, MESC là phần mở rộng của TASD khi hệ thống có thêm dữ liệu metric. MESC bổ sung đối sánh có dấu, tổ hợp nhiều lần chạy và cơ chế đồng thuận để tăng độ ổn định của quyết định lấy mẫu trong điều kiện ngân sách cực thấp.

Tiêu chí	TASD	MESC
Nguồn tín hiệu	Tín hiệu suy ra từ trace	Tín hiệu từ trace kết hợp tín hiệu metric hệ thống
Mức độ nhận biết RCA	Có ở các biến thể mở rộng, thông qua mô hình bốn vùng RCA	Có rõ hơn, thông qua điểm nhận biết RCA và cơ chế đối sánh có dấu
Đa dạng hóa	Dựa trên độ tương đồng Jaccard giữa các đường đi dịch vụ	Dựa trên Jaccard, đồng thời xét thêm độ tin cậy của ánh xạ metric
Tổ hợp	Dựa trên một lần chấm điểm và chọn trace	Tăng độ ổn định bằng cách chạy nhiều hạt giống và lựa chọn thông qua cơ chế đồng thuận
Đồng thuận	Thường dựa trên một lần chấm điểm và chọn trace	Có, dựa trên số phiếu chọn và ngưỡng đồng thuận
Điểm mạnh	Nhẹ, không yêu cầu metric hệ thống đầy đủ	Khai thác thêm bối cảnh vận hành, sắc hơn ở ngân sách rất thấp và giảm phương sai quyết định
Rủi ro	Có thể thiếu tín hiệu hệ thống thật và dễ nhầm trace bất thường với trace hữu ích cho RCA	Chi phí cao hơn, phụ thuộc vào chất lượng metric và cần kiểm soát tốt khi ngân sách tăng

Bảng 3.3: So sánh TASD và MESC.

Bảng 3.3 tóm tắt khác biệt giữa hai phương pháp theo các khía cạnh chính: nguồn tín hiệu đầu vào, mức độ nhận biết RCA, cơ chế đa dạng hóa, khả năng tổ hợp và rủi ro triển khai.

Từ bảng so sánh có thể thấy TASD phù hợp với vai trò phương pháp nền vì nó đơn giản, ít phụ thuộc vào dữ liệu ngoài trace và dễ triển khai trong bối cảnh xử lý dạng luồng. Tuy nhiên, do chỉ sử dụng tín hiệu mức trace, TASD bị giới hạn khi lỗi hệ thống biểu hiện rõ hơn ở metric vận hành. MESC giải quyết hạn chế này bằng cách đưa metric gốc vào scoring và dùng đối sánh có dấu để phân biệt tín hiệu ủng hộ với tín hiệu gây nhiễu. Ngoài ra, sử dụng phương pháp kết hợp và đồng thuận giúp MESC ổn định hơn khi ngân sách lấy mẫu rất nhỏ, nơi mỗi quyết định giữ hoặc loại một trace đều có thể ảnh hưởng lớn đến kết quả MicroRank.

Tuy vậy, MESC không thay thế hoàn toàn TASD. Trong khóa luận, TASD được sử dụng như phương pháp nền để chứng minh hiệu quả của lấy mẫu dựa trên trace, còn MESC được xem là phần mở rộng khi có thêm metric hệ thống và cần tăng độ ổn định ở các mức ngân sách thấp. Cách phân tầng này giúp phương pháp đề xuất vừa có một phương pháp nền gọn nhẹ, vừa có một biến thể nâng cao để khai thác nhiều nguồn tín hiệu hơn.

3.5 Không gian thiết kế và các biến thể mô hình

Các phiên bản sampler trong khóa luận không được trình bày như một danh sách phiên bản rời rạc, mà được đặt trong một không gian thiết kế chung. Cách tổ chức này giúp làm rõ quá trình phát triển phương pháp: từ lấy mẫu chỉ dựa trên trace, đến lấy mẫu nhận biết RCA, sau đó mở rộng bằng metric gốc, tổ hợp nhiều lần chạy và cơ chế đồng thuận.

Không gian thiết kế được mô tả theo ba trục chính: nguồn tín hiệu đầu vào, mục tiêu chấm điểm và cơ chế ổn định quyết định. Ba trục này tương ứng với ba câu hỏi thiết kế cốt lõi: sampler sử dụng loại thông tin nào, sampler tối ưu theo mục tiêu nào, và sampler làm thế nào để giảm dao động quyết định khi ngân sách lấy mẫu rất nhỏ.

3.5.1 Không gian thiết kế

Trục thứ nhất là **nguồn tín hiệu đầu vào**. Ở các phiên bản đầu, sampler chỉ sử dụng các đặc trưng có sẵn ở mức trace như độ trễ, số lượng span, cờ lỗi và đường đi dịch vụ. Cách tiếp cận này giúp phương pháp giữ được tính nhẹ và có thể áp dụng trong bối cảnh không có metric hệ thống đầy đủ. Ở các phiên bản sau, sampler bắt đầu suy ra trạng thái hệ thống từ thống kê trace, rồi tiếp tục mở rộng bằng cách sử dụng metric gốc theo dịch vụ và thời gian.

Trục thứ hai là **mục tiêu chấm điểm**. Đây là trục quan trọng nhất về mặt phương pháp, vì nó thể hiện sự dịch chuyển từ lấy mẫu thiên về bất thường sang lấy mẫu phục vụ RCA. Ban đầu, trace được ưu tiên nếu có lỗi, độ trễ cao hoặc dấu hiệu bất thường mạnh. Tuy nhiên, đối với MicroRank, một trace bất thường chưa chắc đã hữu ích nếu nó không giúp làm nổi bật dịch vụ gây lỗi. Vì vậy, các biến thể sau đưa thêm tư duy đối sánh giữa trace bất thường và trace bình thường, sau đó mở rộng thành đối sánh có dấu khi metric gốc được đưa vào.

Trục thứ ba là **cơ chế ổn định lựa chọn**. Ở ngân sách lấy mẫu rất nhỏ, đặc biệt tại mức 0.1%, việc giữ hoặc loại một trace đơn lẻ có thể ảnh hưởng đáng kể đến kết quả xếp hạng RCA. Do đó, các biến thể sau sử dụng nhiều seed hoặc nhiều thành viên tổ hợp để tạo các quyết định ứng viên, sau đó dùng cơ chế đồng thuận để ưu tiên những trace được nhiều thành viên cùng lựa chọn.

Bảng 3.4 tóm tắt ba trục chính trong không gian thiết kế.

Bảng 3.4: Các trục chính trong không gian thiết kế của phương pháp lấy mẫu.

Trục thiết kế	Các thiết kế chính
Nguồn tín hiệu đầu vào	Dựa trên trace; trạng thái hệ thống suy ra từ trace; metric gốc theo dịch vụ và thời gian.
Mục tiêu chấm điểm	Thiên về bất thường; nhận biết RCA; nhận biết đối sánh; đối sánh có dấu dựa trên metric.
Cơ chế ổn định quyết định	Một lần chạy; tổ hợp nhiều seed; tổ hợp đồng thuận dựa trên bỏ phiếu.

Ba trục thiết kế trên không phát triển độc lập, mà cùng tạo thành một lộ trình chung. Khi nguồn tín hiệu được mở rộng từ trace sang metric, mục tiêu chấm điểm

cũng dịch chuyển từ bất thường đơn lẻ sang đối sánh RCA. Đồng thời, khi ngân sách càng nhỏ, yêu cầu về ổn định quyết định càng lớn, dẫn tới việc sử dụng tổ hợp và đồng thuận.

3.5.2 Các biến thể và cấu hình thực nghiệm

Từ không gian thiết kế trên, khóa luận xác định hai biến thể chính để trình bày và đánh giá thực nghiệm: **TASD** và **MESC**. Hai biến thể này không phải là hai thuật toán cạnh tranh rời rạc, mà đại diện cho hai mức độ hoàn thiện trong cùng một triết lý thiết kế tổng thể, được tóm tắt trong Bảng 3.5.

Bảng 3.5: Các biến thể chính được đánh giá thực nghiệm trong khóa luận.

Tên định danh	Nhóm phương pháp	Vai trò trong khóa luận
TASD	Lấy mẫu thích nghi dựa trên <i>trace</i>	Phương pháp nền; chỉ sử dụng thông tin mức <i>trace</i> , tối ưu hóa theo ràng buộc ngân sách và đảm bảo tính đa dạng của đường đi dịch vụ.
MESC	Lấy mẫu nhận biết RCA kết hợp đồng thuận	Phương pháp mở rộng; tích hợp dữ liệu <i>metric</i> hệ thống, áp dụng đối sánh có dấu và cơ chế đồng thuận để tăng độ ổn định của <i>sampler</i> .

Việc lựa chọn đồng thời cả hai cấu hình này nhằm phục vụ cho các kịch bản triển khai thực tế khác nhau:

Cấu hình TASD được sử dụng để chứng minh tính hiệu quả của phương pháp trong điều kiện thiếu hụt *metric* hệ thống. Đây là một kịch bản phổ biến trong thực tế khi dữ liệu *trace* thường được thu thập đồng bộ hơn so với *metric* chi tiết. Với lợi thế tính toán nhẹ và đơn giản, TASD hoàn toàn phù hợp để triển khai trong môi trường xử lý theo dòng (*streaming*).

Cấu hình MESC được triển khai để đánh giá giới hạn hiệu năng của hệ thống khi có đầy đủ dữ liệu hạ tầng. Khác biệt cốt lõi của MESC nằm ở khả năng sử

dụng cơ chế *đối sánh có dấu* để phân định rõ ràng các tín hiệu ủng hộ và phản bác, kết hợp cùng cơ chế *đồng thuận* giúp triệt tiêu phương sai quyết định khi hoạt động ở các ngưỡng ngân sách cực kỳ khắt khe (ví dụ: 0.1%).

3.6 Tích hợp với MicroRank cho phân tích nguyên nhân gốc

3.6.1 Đầu vào và đầu ra của MicroRank

Đầu vào của MicroRank là tập trace sau lấy mẫu:

$$\mathcal{S}_b = \{t_{i_1}, t_{i_2}, \dots, t_{i_k}\}. \quad (3.21)$$

Trong (3.21), $\mathcal{S}_b$ là tập trace được giữ lại dưới ngân sách lấy mẫu b , còn k là số trace trong tập sau lấy mẫu.

Từ tập trace này, MicroRank sinh ra danh sách dịch vụ được xếp hạng theo mức độ nghi ngờ, như biểu diễn trong Công thức (3.22).

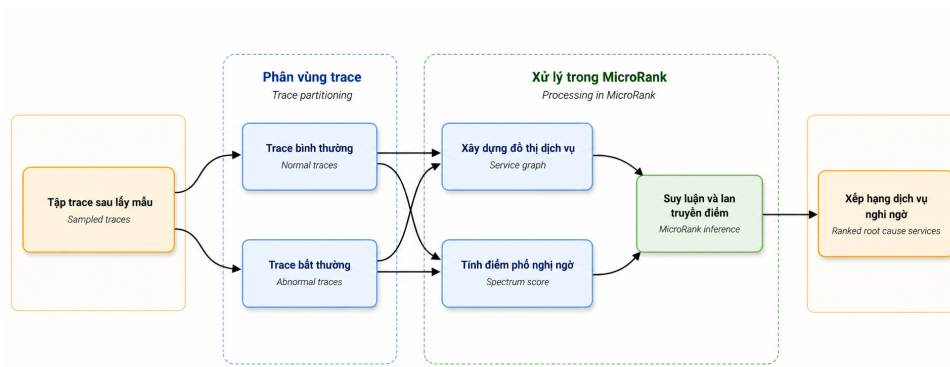
$$R = [(s_1, \text{score}_1), (s_2, \text{score}_2), \dots, (s_m, \text{score}_m)]. \quad (3.22)$$

Trong đó s_1 là dịch vụ được xếp hạng nghi ngờ cao nhất. Nếu s_1 trùng với nguyên nhân gốc rễ thực tế, kịch bản đó đạt $A@1$. Nếu nguyên nhân gốc nằm trong ba dịch vụ đầu, kịch bản đó đạt $A@3$.

MicroRank kết hợp cơ chế lan truyền điểm theo kiểu PageRank [28] với phân tích phổ [3, 27]. Vì vậy, chất lượng của tập trace sau khi lấy mẫu ảnh hưởng trực tiếp đến đồ thị, trọng số cạnh, cách phân chia nhóm bình thường/bất thường và điểm nghi ngờ cuối cùng.

3.6.2 Ảnh hưởng của việc lấy mẫu đến chất lượng RCA

Việc lấy mẫu ảnh hưởng trực tiếp đến chất lượng RCA vì MicroRank không làm việc trên toàn bộ dữ liệu trace gốc, mà trên tập trace còn lại sau khi lấy mẫu. Như minh họa trong Hình 3.5, quyết định giữ lại hay loại bỏ trace có thể làm thay đổi đồ thị dịch vụ, tập dữ liệu bình thường/bất thường, các giá trị thống kê phổ lỗi và kết quả xếp hạng cuối cùng.



Hình 3.5: Ảnh hưởng của tập trace sau lấy mẫu đến MicroRank.

Cụ thể, việc lấy mẫu ảnh hưởng đến MicroRank qua bốn cơ chế chính.

Thứ nhất, quá trình lấy mẫu ảnh hưởng đến đồ thị dịch vụ. Nếu một trace quan trọng bị loại bỏ, liên kết giữa hai dịch vụ có thể sẽ không xuất hiện trong đồ thị sau khi lấy mẫu.

Thứ hai, quá trình lấy mẫu tác động đến việc phân chia tập dữ liệu bình thường và bất thường. Nếu số lượng vết bình thường được giữ lại quá ít, hệ thống MicroRank sẽ không có đủ dữ liệu nền để thực hiện đối sánh. Ngược lại, nếu giữ lại quá nhiều vết lỗi nhiều, tập dữ liệu bất thường sẽ bị pha loãng.

Thứ ba, quá trình lấy mẫu làm thay đổi điểm phổ. Các giá trị thống kê như ef , ep , nf và np sẽ biến thiên phụ thuộc vào tập các vết được giữ lại. Do đó, chỉ cần một vài vết sai lệch cũng có thể làm giảm thứ hạng của dịch vụ là nguyên nhân gốc rễ.

Thứ tư, quá trình lấy mẫu ảnh hưởng trực tiếp đến tính ổn định của bảng xếp hạng. Khi ngân sách lấy mẫu thấp, tập các vết thu được nhỏ sẽ khiến kết quả xếp hạng trở nên nhạy cảm với từng quyết định giữ lại hay loại bỏ. Đây chính là lý do tại sao phương pháp kết hợp đồng thuận lại đóng vai trò vô cùng quan trọng.

Chương 4

Thực nghiệm và Đánh giá

Chương này trình bày toàn bộ quá trình thực nghiệm nhằm kiểm chứng hiệu quả của các phương pháp lấy mẫu thích nghi đã được đề xuất trong Chương 3, bao gồm hai biến thể chính là TASD và MESC. Hướng tiếp cận đánh giá được thiết kế theo dạng đo đặc ngoại tuyến (offline benchmark): tách rời pha lấy mẫu khỏi pha vận hành thời gian thực, tập trung đo và so sánh chất lượng của tập trace giữ lại trên cùng một bộ dữ liệu và cùng một động cơ phân tích nguyên nhân gốc với các phương pháp đã được công bố trong bài báo TraStrainer [14].

Việc tách rời pha lấy mẫu khỏi pha triển khai vận hành cho phép tác giả tập trung vào câu hỏi nghiên cứu cốt lõi: Liệu một bộ lấy mẫu chạy theo dòng, không yêu cầu huấn luyện có giám sát, chỉ giữ lại một tỷ lệ rất nhỏ trace có thể bảo toàn hoặc thậm chí cải thiện chất lượng xếp hạng nguyên nhân gốc của động cơ phân tích phía sau, dưới ràng buộc ngân sách lưu trữ cực thấp hay không?

4.1 Môi trường và dữ liệu thực nghiệm

4.1.1 Cấu hình môi trường và pipeline thực nghiệm

Toàn bộ thực nghiệm được thực hiện trên máy cá nhân với cấu hình tiêu chuẩn, hệ điều hành Windows 10 hoặc Linux. Các script thực hiện được chạy bằng giao diện dòng lệnh, Kết quả trả về được lưu trữ vào các tệp **csv**, **README** để tiện cho việc đối chiếu và so sánh. Về mặt công cụ, môi trường Python được sử dụng phiên bản 3.13.x. Thư viện bắt buộc tối thiểu là **numpy**, do động cơ *MicroRank* được sử dụng ở phần sau cho các phép toán xếp hạng phổ Dstar2 [3, 27].

Việc thực nghiệm trên máy tính cá nhân, không sử dụng các nền tảng đám mây hoặc cụm máy chủ chuyên dụng, phản ánh một thiết lập đánh giá trong môi trường có tài nguyên tính toán hạn chế. Ở mức định hướng, thiết lập này gần với giả định rằng bộ lấy mẫu có thể tích hợp vào các thành phần thu thập trace như Opentelemetry Collector, vốn thường được triển khai như một tiến trình thu thập

dữ liệu chạy kèm với ứng dụng hoặc chạy gần dữ liệu quan sát với tài nguyên giới hạn [29]. Tuy nhiên, cần lưu ý rằng đây vẫn là đánh giá ngoại tuyến. Do đó, kết quả chưa thay thế cho một phép đo triển khai trực tiếp trên môi trường production.

Quy trình hai pha: Quy trình thực nghiệm được tổ chức thành hai pha rõ ràng và tách biệt, minh họa trong Hình ??

1. **Pha lấy mẫu** (*sampling phase*): script run tương ứng với 2 phiên bản, đọc các tệp trace gốc dạng CSV của bộ dữ liệu paper, dựng đối tượng nội bộ `TracePoint` cho mỗi trace, gọi sampler để xác định tập trace được giữ lại, sau đó xuất ra một tập trace mới đã được lấy mẫu trên đĩa.
2. **Pha RCA** (*root cause analysis phase*): tập trace là kết quả của pha lấy mẫu được đưa vào MicroRank engine để sinh bảng xếp hạng dịch vụ nghi ngờ. Kết quả sinh ra được chuẩn hoá về một định dạng đầu ra và được đối chiếu với nhãn ground-truth để tính các chỉ số $A@1$, $A@3$ và MRR .

Toàn bộ pipeline được điều phối bởi hai phần chính. Nhóm runner thứ nhất, `run_paper_sampled_rca.py` dùng cho nhóm sampler nền, tương ứng với phương pháp TASD. `run_paper_sampled_rca_v9_fast.py` dùng cho sampler chính, tương đương với biến thể MESC. Đặc biệt, runner được tối ưu chạy nhanh hơn nhờ caching và song song hóa pha RCA.

Mặc dù được thực hiện trên môi trường ngoại tuyến, pipeline thực nghiệm không chỉ mô phỏng việc lấy mẫu dựa trên danh sách **ID** của trace. Sau khi bộ lấy mẫu chọn ra tập trace cần giữ lại, các trace này được chuẩn hóa thành đầu vào phù hợp và được đưa trực tiếp vào MicroRank. Vì vậy kết quả đo được phản ánh đúng mức độ ảnh hưởng thực tế của bộ lấy mẫu đối với động cơ RCA phía sau, bao gồm cả tác động lên đồ thị dịch vụ, phân vùng trace cũng như giá trị thống kê phổ lỗi.

4.1.2 Bộ dữ liệu thực nghiệm

Khoá luận sử dụng **Dataset B** của bài báo *TraStrainer*, là một bộ dữ liệu chuẩn trong nghiên cứu trace-based RCA. Bộ dữ liệu bao gồm ba nhóm con thu thập từ hai hệ vi dịch vụ phổ biến trong cộng đồng học thuật [14, 19, 30]. Bảng 4.1 trình bày tổng quan kích thước của ba bộ dữ liệu, thể hiện sự khác biệt đáng kể về số lượng trace, span và số kịch bản sự cố.

Một lưu ý khi sử dụng **Dataset B** là số lượng trace thu được từ thư mục dữ liệu công khai trên **github** của *TraStrainer* trong quá trình tái lập thực nghiệm lớn hơn số lượng được báo cáo tổng quan trong bài báo gốc. Cụ thể, trong bài báo, nhóm nhà nghiên cứu công bố thông tin về **Dataset B** gồm 112,000 trace, 32 loại metric, 56 batch và 5 loại lỗi [14]. Tuy nhiên khi lấy dữ liệu về và thực hiện kiểm tra sơ bộ trước khi đưa vào thực nghiệm, kết quả được tổng hợp trong Bảng 4.1 số lượng trace thực tế ghi nhận được cao hơn thống kê trong bài báo gốc [31].

Sự khác biệt này không ảnh hưởng đáng kể đến phép so sánh trong khóa luận, vì tất cả các phương pháp được đánh giá trên cùng tập dữ liệu thực tế sau khi trích xuất, cùng tập kịch bản sự cố và cùng một quy trình RCA ở bước sau. Việc dữ liệu thực nghiệm có quy mô lớn hơn cũng làm cho phép đánh giá có độ bao phủ rộng hơn, do bộ lấy mẫu phải chọn một tỷ lệ trace rất nhỏ từ không gian đầu vào rộng hơn. Vì vậy, nếu phương pháp vẫn duy trì được các chỉ số $A@1$, $A@3$ và MRR ở mức cạnh tranh, kết quả này cung cấp thêm bằng chứng thực nghiệm chứng minh rằng phương pháp có khả năng giữ lại các trace quan trọng cho RCA ngay cả khi quy mô trace đầu vào tăng lên.

Bảng 4.1: Tổng quan ba bộ dữ liệu thực nghiệm.

Bộ dữ liệu	Số kịch bản	Tổng số trace	Tổng số span
train-ticket	17	27,364	1,044,580
hipster-batch1	24	121,449	5,474,387
hipster-batch2	32	177,945	8,059,526
Tổng	73	326,758	14,578,493

Mỗi bộ dữ liệu được tổ chức theo cấu trúc thư mục:

- **label.json**: tệp ground-truth chứa thông tin về thời điểm sự cố, dịch vụ là nguyên nhân gốc, và mã định danh của từng kịch bản.
- **trace/**: dữ liệu trace gốc, mỗi tệp CSV tương ứng với một kịch bản, ghi nhận các span được sinh ra trong cửa sổ sự cố.
- **metric/**: dữ liệu metric mức hệ thống (CPU, memory, latency, throughput, success rate) được lấy mẫu theo chu kỳ giây, tổ chức theo từng pod / dịch vụ.

4.1.3 Biểu diễn trace và trích xuất kịch bản sự cố

Trong pha lấy mẫu, mỗi trace gốc được chuyển thành một biểu diễn **TracePoint** rút gọn ở mức trace, bao gồm các thông tin như định danh trace, thời điểm bắt đầu, thời lượng, số lượng span, trạng thái lỗi và tập dịch vụ xuất hiện trong trace. Biểu diễn này được sử dụng làm đầu vào chung cho các bộ lấy mẫu, giúp quá trình chấm điểm trace diễn ra nhẹ hơn và thống nhất giữa các phương pháp.

Mỗi kịch bản sự cố được xây dựng từ tệp `label.json`, trong đó nhãn sự cố được dùng để xác định thời điểm xảy ra lỗi và dịch vụ nguyên nhân gốc. Cửa sổ quan sát của mỗi kịch bản được thiết lập gồm 300 giây trước thời điểm sự cố và 600 giây sau thời điểm sự cố. Cách thiết lập này giúp mỗi kịch bản bao gồm cả ngữ cảnh trước lỗi và các trace phát sinh sau khi lỗi xảy ra.

Đầu ra của bước này là các tệp JSONL có cấu trúc chuẩn, trong đó mỗi dòng tương ứng với một kịch bản sự cố và chứa các thông tin như cửa sổ thời gian, dịch vụ nguyên nhân gốc, cùng đường dẫn tới dữ liệu trace và metric tương ứng. Các tệp JSONL này đóng vai trò *nguồn dữ liệu thống nhất* cho cả pha lấy mẫu và pha đánh giá RCA, qua đó bảo đảm tính nhất quán và khả năng tái lập của thực nghiệm.

4.2 Phương pháp và chỉ số đánh giá

Do quy trình thực nghiệm cố định MicroRank làm động cơ RCA ở pha sau, phần đánh giá tập trung vào hai khía cạnh chính: chất lượng RCA của tập trace sau lấy mẫu và chi phí của quá trình lấy mẫu. Cách thiết lập này giúp so sánh tác động của từng phương pháp lấy mẫu trong khi giữ nguyên dữ liệu đầu vào, tập kịch bản sự cố và động cơ RCA.

Chỉ số chất lượng RCA. Chất lượng RCA được đánh giá bằng ba chỉ số chính gồm **A@1**, **A@3** và **MRR**, đã được định nghĩa ở Mục 2.3.4. Trong chương thực nghiệm, các chỉ số này được tính riêng cho từng bộ dữ liệu, từng mức ngân sách lấy mẫu và từng phương pháp được so sánh. Khi báo cáo kết quả tổng hợp, khóa luận sử dụng trung bình có trọng số theo số kịch bản của từng bộ dữ liệu.

Chỉ số chi phí thời gian. Bên cạnh chất lượng RCA, khóa luận ghi nhận chi phí thời gian của quá trình lấy mẫu, gồm thời gian thực thi lõi của sampler, thời gian áp dụng giới hạn ngân sách, tổng thời gian lựa chọn trace và thời gian xử lý trung

binh trên mỗi trace. Đối với các phương pháp sử dụng thêm metric, thời gian tiền xử lý metric cũng được ghi nhận riêng để phân biệt chi phí chuẩn bị dữ liệu với chi phí chấm điểm trace.

Chỉ số phụ chất lượng lấy mẫu. Ngoài hai nhóm chỉ số trên, khóa luận còn ghi nhận một số chỉ số phụ như tỷ lệ trace lỗi, span, tỉ lệ trace nằm trong giai đoạn sớm của sự cố được giữ lại. Các chỉ số này chủ yếu được dùng để hỗ trợ phân tích kết quả, không phải tiêu chí chính để kết luận phương pháp tốt hơn.

4.3 Cài đặt các chiến lược lấy mẫu

4.3.1 Ràng buộc về ngân sách

Mỗi phương pháp lấy mẫu được đánh giá tại ba mức ngân sách lưu trữ: **0.1%**, **1.0%** và **2.5%**. Các mức ngân sách này được lựa chọn để tương ứng với thiết lập thực nghiệm trong bài báo *TraStrainer*, đồng thời phản ánh ba mức nén dữ liệu khác nhau [14]. Trong đó, **0.1%** là mức nén cực mạnh, **1.0%** là mức tham chiếu trung bình, còn **2.5%** là mức ngân sách nới lỏng hơn để đánh giá hành vi của bộ lấy mẫu khi số trace được giữ lại tăng lên.

Để bảo đảm so sánh công bằng, toàn bộ phương pháp được chạy ở chế độ *strict cap*. Với mỗi kịch bản sự cố, số trace được giữ lại được ép đúng theo ngân sách mục tiêu. Nếu bộ lấy mẫu chọn nhiều hơn số lượng mục tiêu, các trace có điểm thấp hơn sẽ bị loại bỏ ở bước cuối; nếu chọn ít hơn, các trace có điểm cao kế tiếp sẽ được bổ sung. Cơ chế này loại trừ khả năng một phương pháp đạt kết quả tốt hơn chỉ vì giữ lại nhiều trace hơn các phương pháp còn lại.

Ở mức ngân sách **0.1%**, số trace được giữ lại rất nhỏ so với tập trace ban đầu. Khi đó, việc thay đổi một vài trace trong tập được giữ lại cũng có thể làm thay đổi đồ thị dịch vụ, phân vùng trace bình thường/bất thường và kết quả xếp hạng của MicroRank. Vì vậy, mức ngân sách này được xem là thiết lập khắc nghiệt nhất để kiểm tra khả năng giữ lại các trace có ích cho RCA.

4.3.2 Các phương pháp đối chứng

Để đặt phương pháp đề xuất trong bối cảnh so sánh với các nghiên cứu trước, khóa luận sử dụng các nhóm phương pháp đối chứng được báo cáo trong bài báo

TraStrainer [14]. Nhóm thứ nhất là các phương pháp lấy mẫu truyền thống như *Random Sampling* và *Head-based Sampling*. Nhóm thứ hai là các phương pháp lấy mẫu dựa trên đặc trưng như *Sieve* [16] và *Sifter* [13]. Nhóm thứ ba là các biến thể của *TraStrainer*, bao gồm mô hình đầy đủ và các phiên bản loại bỏ một số thành phần chính.

Các kết quả của nhóm bộ lấy mẫu truyền thống và *TraStrainer* được lấy nguyên văn từ bài báo gốc để đảm bảo tính nhất quán. Các bộ lấy mẫu đề xuất nội bộ được chạy lại trên cùng pipeline với cùng cấu hình ngân sách.

4.3.3 Cấu hình các phương pháp đề xuất

Khóa luận đánh giá hai phương pháp lấy mẫu chính đã trình bày trong Chương 3, gồm T ASD và MESC. T ASD được sử dụng như phương pháp nền nội bộ, đại diện cho hướng lấy mẫu thích nghi chỉ dựa trên thông tin mức trace. Trong khi đó, MESC là phương pháp đề xuất chính, mở rộng T ASD bằng cách bổ sung tín hiệu metric gốc và cơ chế đối sánh có dấu nhằm hỗ trợ tốt hơn cho pha phân tích nguyên nhân gốc.

Trong thực nghiệm, T ASD chỉ sử dụng các đặc trưng rút gọn từ trace như độ trễ, trạng thái lỗi, dịch vụ xuất hiện trong trace và quan hệ đường đi dịch vụ. Ngược lại, MESC khai thác thêm dữ liệu metric mức hệ thống, bao gồm CPU, bộ nhớ, độ trễ, lưu lượng và tỷ lệ thành công. Các chi tiết về cách xây dựng điểm lấy mẫu, cơ chế đa dạng hóa và đối sánh có dấu đã được trình bày trong Mục 3.2 và Mục 3.4.

Cả hai phương pháp được chạy trên cùng tập dữ liệu, cùng tập kịch bản sự cố, cùng các mức ngân sách lấy mẫu và cùng động cơ RCA ở pha sau. Cách thiết lập này cho phép đánh giá trực tiếp tác động của việc bổ sung metric gốc và cơ chế đối sánh có dấu đối với chất lượng RCA.

4.4 Kết quả thực nghiệm

Quy ước: Mỗi lần chạy thực nghiệm sinh ra các tệp báo cáo tổng hợp theo từng bộ dữ liệu, từng mức ngân sách và từng phương pháp lấy mẫu. Trong chương này, các giá trị $A@1$ và $A@3$ được biểu diễn dưới dạng phần trăm, còn MRR là giá trị không có đơn vị trong khoảng $[0, 1]$.

Phương pháp	A@1 (%)			A@3 (%)			MRR		
	0.1%	1.0%	2.5%	0.1%	1.0%	2.5%	0.1%	1.0%	2.5%
Random	5.56	16.67	27.78	20.37	50.00	61.11	0.1571	0.3423	0.4352
HC	7.41	18.52	22.22	27.78	46.30	51.85	0.1954	0.3398	0.3731
Sifter	5.56	18.52	27.78	23.42	46.30	61.11	0.1605	0.3414	0.4358
Sieve	9.26	25.83	35.19	20.37	58.15	62.96	0.1657	0.4246	0.4963
TraStrainer w/o M	12.96	16.67	24.07	42.59	42.59	55.56	0.2994	0.3241	0.4012
TraStrainer w/o D	29.63	42.59	46.30	74.04	68.52	72.22	0.5228	0.5463	0.5509
TraStrainer	42.59	45.16	50.00	77.74	78.52	82.26	0.5509	0.5889	0.6556
TASD	23.37	26.12	18.50	46.49	56.41	53.29	0.4105	0.4443	0.3988
MESC	41.30	17.93	13.77	48.82	44.53	35.05	0.5168	0.3685	0.3200

Bảng 4.2: So sánh TASD và MESC với các phương pháp lấy mẫu khác trên ba mức ngân sách.

4.4.1 Đánh giá kết quả của TASD

Bảng 4.2 trình bày kết quả của TASD khi đối chiếu với các phương pháp lấy mẫu trong bài báo *TraStrainer*. Các số liệu được tổng hợp theo ba mức ngân sách 0.1%, 1.0% và 2.5%. Trong đó, các phương pháp như Random, HC, Sifter, Sieve và TraStrainer được dùng làm nhóm đối chứng, còn TASD là phương pháp lấy mẫu thích nghi nền của khóa luận.

So sánh với nhóm sampler truyền thống.

Trong Bảng 4.2, TASD cho thấy hiệu quả rõ rệt ở mức ngân sách thấp 0.1%. Tại mức này, TASD đạt $A@1 = 23.37\%$, cao hơn đáng kể so với Random, HC, Sifter và Sieve. Cụ thể, TASD cao hơn Random (5.56%) khoảng 4.2 lần, cao hơn Sifter (5.56%) và Sieve (9.26%) tương ứng 4.2 và 2.5 lần. Chỉ số MRR của TASD đạt 0.4105, cũng cao gần gấp 2.6 lần so với Random (0.1571) và gấp khoảng 2.5 lần so với Sieve (0.1657). Đáng chú ý, TASD cũng vượt Sieve, dù Sieve sử dụng **robust random cut forest** để ưu tiên các trace bất thường [16]. Kết quả này chứng minh cho việc chỉ tập trung vào anomaly là chưa đủ cho bài toán RCA.

So sánh với các biến thể của TraStrainer.

Khi so với TraStrainer w/o M – biến thể chỉ dùng trace diversity, có sử dụng metric nhưng không quan tâm tới thiên lệch hệ thống, TASD đạt kết quả vượt trội ở các mức ngân sách thấp trên cả ba chỉ số. Cụ thể, ở mức 0.1%, TASD đạt $A@1 = 23.37\%$ so với 12.96% của TraStrainer w/o M, chênh lệch khoảng 10.4 điểm phần trăm. Ở mức 1.0%, TASD đạt 26.12% so với 16.67%, vẫn duy trì khoảng cách khoảng 9.5 điểm. Sự chênh lệch này phản ánh khả năng khai thác tín hiệu mức trace hiệu quả hơn của TASD với biến thể TraStrainer w/o M. Một phần khác có thể đến từ việc TASD bổ sung thêm tín hiệu độ gần của sở sự cố và mô hình bốn vùng RCA trong các biến thể mở rộng, vốn không có trong thiết kế gốc của TraStrainer w/o M.

Tuy nhiên, TASD vẫn còn khoảng cách đáng kể so với TraStrainer w/o D và TraStrainer đầy đủ. Khoảng cách này không bất ngờ vì hai biến thể trên đều sử dụng thông tin metric hệ thống, vốn cung cấp tín hiệu trực tiếp về trạng thái vận hành của các dịch vụ. Đây cũng chính là động cơ dẫn đến việc đề xuất MESC ở phần tiếp theo của khóa luận.

Phân tích đường cong theo ngân sách

Một quan sát đáng chú ý là chất lượng của TASD không tăng đơn điệu theo ngân sách. $A@1$ tăng từ 23.37% ở mức 0.1% lên 26.12% ở mức 1.0%, nhưng giảm xuống 18.50% ở mức 2.5%. Xu hướng tương tự xuất hiện ở MRR , khi chỉ số này đạt cực đại 0.4443 tại 1.0% rồi giảm còn 0.3988 tại 2.5%.

Hiện tượng này cũng cung cấp bằng chứng cho việc giữ thêm trace không luôn cải thiện chất lượng RCA. Khi ngân sách tăng, sampler có thể giữ thêm các trace có điểm thấp hơn, bao gồm trace nhiễu hoặc trace trùng lặp về đường đi dịch vụ. Các trace này có thể làm giảm độ tương phản phổ lỗi trong MicroRank và khiến dịch vụ nguyên nhân gốc bị xếp thấp hơn. Do đó, lấy mẫu cho RCA không chỉ là bài toán tăng độ phủ trace, mà còn là bài toán cân bằng giữa độ phủ tín hiệu và độ sạch của tập trace sau lấy mẫu.

Phân tích chỉ số $A@3$ so với $A@1$

Một góc nhìn khác là so sánh giữa $A@1$ và $A@3$. Ở mức 0.1%, $A@3$ của T ASD đạt 46.49%, gần gấp đôi $A@1$ (23.37%). Tỷ lệ này cao hơn so với Random (gấp 3.7 lần) và HC (gấp 3.7 lần). Có thể nói, T ASD ổn định hơn trong việc đưa nguyên nhân gốc vào nhóm ba dịch vụ nghi ngờ hàng đầu. Ở mức 1.0%, khoảng cách $A@3 - A@1$ của T ASD là 30.29 điểm phần trăm, cao hơn so với hầu hết các baseline trừ TraStrainer. Mức chênh lệch lớn giữa $A@1$ và $A@3$ cho thấy T ASD thường đưa nguyên nhân gốc vào danh sách nghi ngờ với độ tin cậy cao, nhưng chưa luôn đặt nó ở vị trí cao nhất. Trong bối cảnh vận hành thực tế, $A@3$ thường được xem là chỉ số có ý nghĩa thực tiễn cao hơn $A@1$, vì người vận hành có thể kiểm tra ba dịch vụ nghi ngờ hàng đầu thay vì chỉ một. Vì vậy, kết quả $A@3 = 56.41\%$ ở mức 1.0% của T ASD vẫn có giá trị thực tiễn, mặc dù $A@1$ chỉ đạt 26.12%.

Tổng hợp về T ASD.

Tổng kết lại, T ASD đã chứng minh được giá trị của hướng tiếp cận chấm điểm theo luật kinh nghiệm cho lấy mẫu trace nhận biết RCA. Nhờ kết hợp chấm điểm trace theo luật kinh nghiệm, kiểm soát đa dạng đường đi dịch vụ và ràng buộc tỷ lệ trace lỗi/bình thường, T ASD phần nào khắc phục được hạn chế của các sampler thiên về bất thường thuần túy. Ngay cả khi không sử dụng metric hệ thống, các tín hiệu mức trace như độ trễ, cờ lỗi, độ gần cửa sổ sự cố và cấu trúc đường đi dịch vụ vẫn giúp T ASD. Tuy nhiên, hai hạn chế đã được làm rõ qua thực nghiệm.

Thứ nhất, do chỉ sử dụng thông tin mức trace, T ASD không thể tận dụng các tín hiệu vận hành quan trọng như metric hệ thống, dẫn đến khoảng cách rõ rệt so với TraStrainer.

Thứ hai, T ASD chưa có cơ chế kiểm soát mở rộng tốt khi ngân sách tăng, biểu hiện qua hiện tượng suy giảm chất lượng ở mức 2.5%.

Hai hạn chế này chính là hai trục cải tiến mà MESC nhắm tới: bổ sung metric gốc kết hợp với đối sánh có dấu để cải thiện chất lượng tín hiệu, và sử dụng cơ chế đồng thuận tổ hợp để tăng độ ổn định của quyết định lấy mẫu trên các mức ngân sách khác nhau.

4.4.2 Đánh giá kết quả của MESC

Bảng 4.2 trình bày kết quả của MESC khi đối chiếu với các phương pháp lấy mẫu trong bài báo *TraStrainer*. MESC là phương pháp đề xuất chính của khóa luận, sử dụng thêm metric gốc và cơ chế đối sánh có dấu để chọn các trace hỗ trợ tốt hơn cho phân tích RCA.

Hiệu năng ở mức ngân sách cực thấp

Quan sát Bảng 4.2, MESC đạt kết quả tốt nhất ở mức ngân sách 0.1%, với $A@1 = 41.30\%$, $A@3 = 48.82\%$ và $MRR = 0.5168$. So với *TraStrainer* đầy đủ, MESC chỉ thấp hơn 1.29 điểm phần trăm về $A@1$ và chênh lệch 0.0341 về MRR . Đây là kết quả đáng chú ý vì MESC không yêu cầu huấn luyện mô hình, mà dựa trên metric gốc, đối sánh có dấu và cơ chế đồng thuận.

So với T ASD, MESC cải thiện $A@1$ từ 23.37% lên 41.30%, đồng thời tăng MRR từ 0.4105 lên 0.5168. Điều này chứng minh rằng metric gốc và đối sánh có dấu giúp bộ lấy mẫu phân biệt tốt hơn các trace có giá trị cho RCA, đặc biệt trong điều kiện ngân sách cực thấp, nơi mỗi quyết định giữ hoặc loại trace đều ảnh hưởng lớn đến kết quả xếp hạng cuối cùng.

So sánh với các sampler thiên về phân tích dị thường thuần túy.

Một góc nhìn quan trọng là so sánh MESC với Sieve, một phương pháp được thiết kế chuyên biệt để thiên lệch về phía edge-case traces [16]. Ở mức 0.1%, MESC đạt $A@1 = 41.30\%$ trong khi Sieve chỉ đạt 9.26%, chênh lệch khoảng 32 điểm phần trăm hay gấp khoảng 4.5 lần. Khoảng cách lớn này không thể chỉ giải thích bằng việc MESC sử dụng thêm metric, mà còn phản ánh sự khác biệt cơ bản trong triết lý thiết kế. Sieve tối ưu việc giữ lại trace bất thường nhưng không kiểm soát thành phần của tập trace được giữ. Khi đưa vào MicroRank, một tập gồm chủ yếu trace lỗi mà thiếu trace bình thường sẽ làm phổ lỗi mất độ tương phản, vì hầu hết các dịch vụ đều có ef cao và ep thấp, dẫn đến điểm nghi ngờ phân tán đều. Ngược lại, MESC với mô hình bốn vùng RCA chủ động duy trì cân bằng giữa các trace ở vùng Vàng, Bạc, Pha trộn và Nhiều, qua đó bảo toàn được cấu trúc tín hiệu phổ lỗi cần thiết cho MicroRank.

Hiện tượng suy giảm khi ngân sách tăng.

Tuy nhiên, MESC bộc lộ một hiện tượng đáng chú ý là đường cong chất lượng giảm mạnh khi ngân sách tăng. Hiện tượng này có thể được giải thích từ ba nguồn nguyên nhân chính.

Thứ nhất, cơ chế đồng thuận tổ hợp ưu tiên các trace được nhiều seed cùng lựa chọn, giúp ổn định quyết định ở ngân sách thấp. Tuy nhiên, khi ngân sách tăng, bộ lấy mẫu phải bổ sung thêm các trace có mức đồng thuận thấp hơn. Đây thường là các trace có điểm gần ngưỡng, ít được các seed thống nhất và dễ trở thành nguồn nhiễu cho MicroRank.

Thứ hai, đối sánh có dấu trong MESC tạo ra phân phối điểm lệch mạnh: một số trace có bằng chứng metric rõ ràng nhận điểm rất cao, trong khi phần lớn trace còn lại có điểm thấp. Ở ngân sách thấp, bộ lấy mẫu chủ yếu chọn phần đỉnh của phân phối. Và khi ngân sách tăng, bộ lấy mẫu lấy thêm trace từ vùng điểm thấp hơn, nơi tín hiệu đối sánh kém rõ ràng và dễ bị nhiễu bởi các biến động metric không liên quan đến nguyên nhân gốc.

Thứ ba, các ràng buộc về tỷ lệ trace lỗi/bình thường chủ yếu phát huy hiệu quả ở ngân sách thấp. Ở mức 0.1%, các ngưỡng tối thiểu giúp bảo đảm tập trace giữ lại đủ dữ liệu lỗi và bình thường cho MicroRank đối sánh. Tuy nhiên, khi ngân sách tăng lên 1.0% hoặc 2.5%, các ràng buộc này không còn đủ chặt để kiểm soát chất lượng phần trace bổ sung, khiến tập trace cuối cùng có thể mất cân bằng hoặc chứa nhiều trace dư thừa.

Phân tích chỉ số $A@3$ so với $A@1$.

Ở mức 0.1%, $A@3$ của MESC đạt 48.82%, chỉ cao hơn $A@1$ (41.30%) khoảng 7.5 điểm phần trăm. Tỷ lệ $A@3/A@1 = 1.18$ này thấp hơn rõ rệt so với T ASD (1.99) và TraStrainer (1.83). Quan sát này gợi ý rằng khi MESC đã xác định được nguyên nhân gốc, nó có xu hướng đặt ngay ở vị trí số một thay vì phân tán trong top-3. Đây là biểu hiện của một bộ lấy mẫu "quyết đoán": một khi metric gốc và đối sánh có dấu cho ra bằng chứng mạnh, nguyên nhân gốc thường được đẩy lên top-1 trong xếp hạng của MicroRank. Tuy nhiên, mặt trái là khi metric gốc cung cấp tín hiệu sai hoặc gây nhiễu, MESC có thể bỏ lỡ nguyên nhân gốc hoàn toàn (cả trong top-3), điều này phản ánh qua sự sụt giảm song song của cả $A@1$ và $A@3$ ở

các mức ngân sách cao hơn.

4.4.3 Phân tích theo từng bộ dữ liệu

Bảng 4.3 trình bày kết quả của T ASD và MESC trên từng bộ dữ liệu. Thể hiện hiệu quả của hai phương pháp không đồng nhất giữa các workload, đặc biệt khi so sánh giữa train-ticket và hai bộ hipster.

Bảng 4.3: Kết quả $A@1$, $A@3$ và MRR của T ASD và MESC trên từng bộ dữ liệu.

Bộ dữ liệu	Ngân sách	T ASD			MESC		
		$A@1$	$A@3$	MRR	$A@1$	$A@3$	MRR
train-ticket	0.1%	0.1176	0.3529	0.3142	0.1765	0.2353	0.2720
	1.0%	0.2941	0.5882	0.4520	0.0588	0.2941	0.2282
	2.5%	0.1176	0.5882	0.3591	0.0588	0.1765	0.1960
hipster-batch1	0.1%	0.3333	0.5417	0.4856	0.5000	0.5417	0.6058
	1.0%	0.2083	0.5417	0.4149	0.2917	0.5417	0.4670
	2.5%	0.2500	0.5417	0.4432	0.1667	0.5000	0.3910
hipster-batch2	0.1%	0.2500	0.5000	0.4315	0.5625	0.6875	0.6727
	1.0%	0.2812	0.5625	0.4661	0.1875	0.5000	0.4103
	2.5%	0.1875	0.4688	0.3943	0.1875	0.3750	0.3731

Ở mức ngân sách 0.1%, MESC cải thiện rõ rệt trên hai bộ hipster. Trên hipster-batch1, $A@1$ tăng từ 0.3333 lên 0.5000; trên hipster-batch2, $A@1$ tăng từ 0.2500 lên 0.5625, đồng thời MRR cũng tăng mạnh từ 0.4315 lên 0.6727. Điều này cho thấy metric gốc và đối sánh có dấu đặc biệt hữu ích khi tín hiệu metric phản ánh rõ trạng thái bất thường của workload.

Ngược lại, trên train-ticket, MESC không vượt T ASD một cách ổn định. Mặc dù $A@1$ ở mức 0.1% cao hơn T ASD, các chỉ số $A@3$ và MRR lại thấp hơn; Trong khi ở hai mức 1.0% và 2.5%, T ASD cho kết quả tốt hơn trên cả ba chỉ số. Kết quả thể hiện rằng hiệu quả của MESC phụ thuộc vào chất lượng và mức độ phù hợp của tín hiệu metric đối với từng bộ dữ liệu.

Nhìn chung, phân tích chỉ ra rằng MESC có ưu thế rõ nhất ở ngân sách cực thấp và trên các workload có tín hiệu metric phân biệt tốt. Trong khi đó, T ASD ổn định hơn ở trường hợp ngân sách cao hơn, do không phụ thuộc trực tiếp vào metric gốc.

4.4.4 So sánh chi phí thời gian

Bên cạnh chất lượng RCA, khóa luận cũng đánh giá chi phí thời gian của quá trình lấy mẫu. Bảng 4.4 trình bày thời gian xử lý của **MESC** trên ba bộ dữ liệu và ba mức ngân sách. Các chỉ số chính gồm thời gian xử lý lỗi, chi phí lỗi trên mỗi trace và tổng chi phí trên mỗi trace.

Bảng 4.4: Chi phí thời gian của sampler trên ba bộ dữ liệu và ba mức ngân sách.

Dataset	Budget	Total traces	Kept traces	Kept ratio (%)	Core time (s)	Core ms/trace	Total ms/trace
train-ticket	0.1%	27,364	27	0.0987	20.94	0.7654	0.8245
train-ticket	1.0%	27,364	274	1.0013	21.11	0.7715	0.8306
train-ticket	2.5%	27,364	684	2.4996	21.89	0.8001	0.8592
hipster-batch1	0.1%	121,449	121	0.0996	237.75	1.9576	2.0064
hipster-batch1	1.0%	121,449	1,214	0.9996	248.45	2.0457	2.0945
hipster-batch1	2.5%	121,449	3,036	2.4998	263.78	2.1719	2.2206
hipster-batch2	0.1%	177,945	178	0.1000	502.90	2.8261	2.8756
hipster-batch2	1.0%	177,945	1,779	0.9997	521.55	2.9309	2.9804
hipster-batch2	2.5%	177,945	4,449	2.5002	570.18	3.2043	3.2537

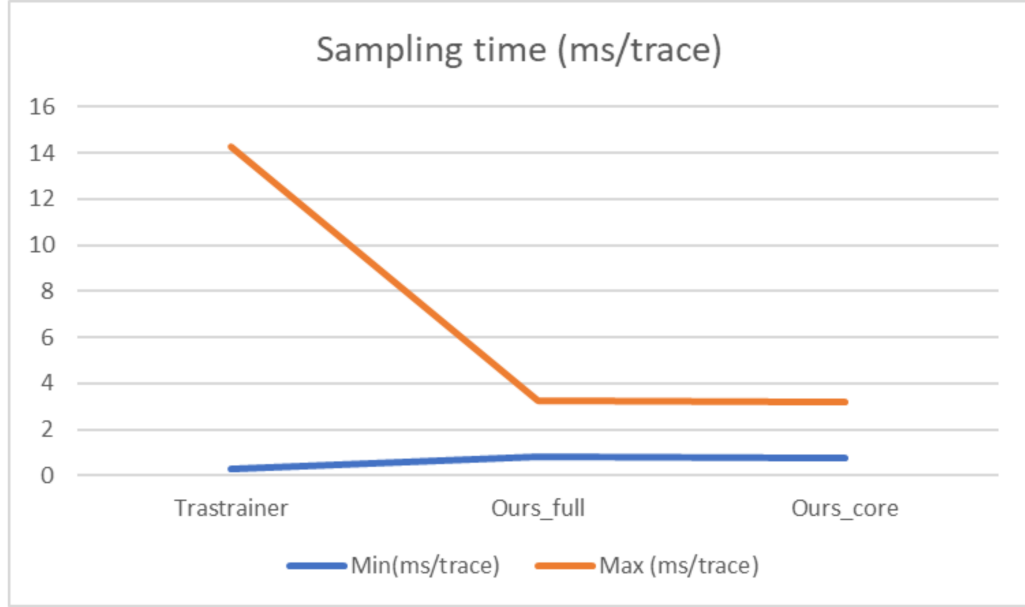
Quan sát Bảng 4.4, chi phí xử lý lỗi của sampler dao động từ 0.7654 ms/trace đến 3.2043 ms/trace. Nếu tính cả chi phí lựa chọn và tiền xử lý được phân bổ, tổng chi phí dao động từ khoảng 0.8245 ms/trace đến 3.2537 ms/trace. Trên train-ticket, tổng chi phí chỉ khoảng 0.82–0.86 ms/trace, trong khi trên hai bộ hipster lớn hơn, chi phí tăng lên khoảng 2.01–3.25 ms/trace. Sự khác biệt này phản ánh ảnh hưởng của quy mô dữ liệu và độ phức tạp của trace đến quá trình chấm điểm.

Khi so sánh tham chiếu với TraStrainer, bài báo gốc báo cáo độ trễ xử lý trong khoảng 0.28–14.29 ms/trace [14]. Kết quả của MESC nằm trong cùng miền độ lớn và thấp hơn mức cực đại được báo cáo của TraStrainer. Tuy nhiên, đây không phải là so sánh tốc độ tuyệt đối, vì hai phép đo có thể khác nhau về môi trường thực nghiệm, cấu hình phần cứng và cách triển khai. Điểm đáng chú ý là MESC đạt chi phí xử lý thấp mà không yêu cầu huấn luyện mô hình dự đoán metric.

Khi ngân sách tăng từ 0.1% lên 2.5%, chi phí xử lý trên mỗi trace chỉ tăng nhẹ trong từng bộ dữ liệu. Sự thay đổi này gợi ý rằng thời gian xử lý của MESC tương đối ổn định theo ngân sách trong phạm vi thực nghiệm. Nguyên nhân là phần lớn chi phí đến từ bước quét toàn bộ trace và tính điểm cho từng trace, trong khi số lượng trace được giữ lại chỉ ảnh hưởng đến các bước sau như sắp xếp, chặn ngân sách và ghi tập trace kết quả. Tính trên toàn bộ các phép đo trong Bảng 4.4, sampler xử lý tổng cộng 980,274 lượt trace và giữ lại 11,762 trace. Chi phí trung bình có trọng

số xấp xỉ 2.46 ms/trace cho phần xử lý lỗi và 2.51 ms/trace nếu tính tổng chi phí. Điều này phản ánh rằng MESC có chi phí xử lý ổn định theo ngân sách và vẫn duy trì khả năng kiểm soát số lượng trace được giữ lại.

So sánh khoảng phân tán thời gian giữa các sampler



Hình 4.1: So sánh khoảng Min–Max thời gian lấy mẫu trên mỗi trace giữa TraStrainer và MESC.

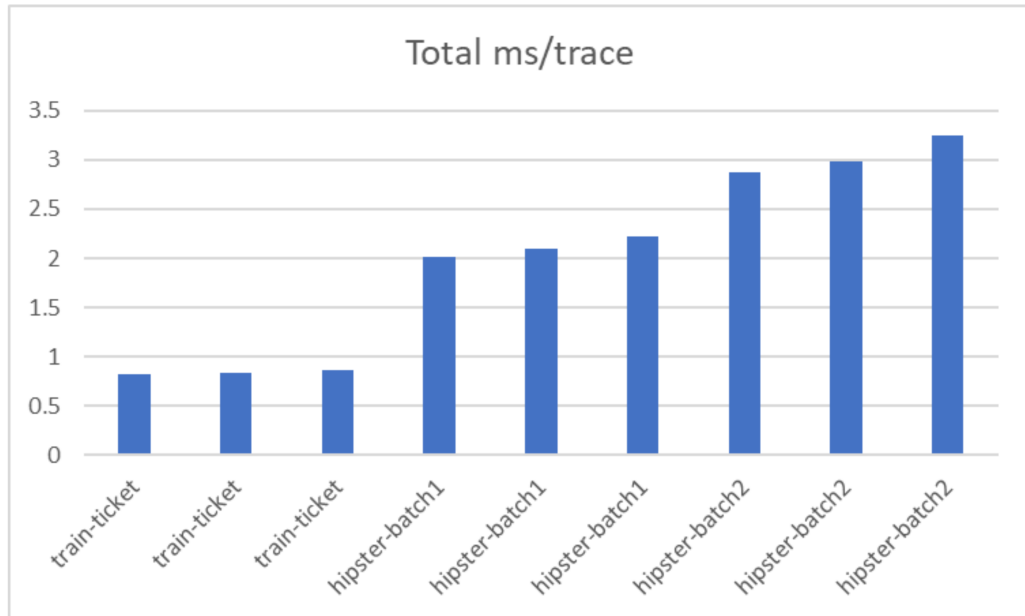
Để đánh giá độ ổn định của thời gian lấy mẫu, Hình 4.1 trình bày khoảng giá trị nhỏ nhất và lớn nhất của chi phí xử lý trên mỗi trace giữa TraStrainer và MESC. Trong đó, *MESC-full* tính cả chi phí tiền xử lý metric đã được phân bổ, còn *MESC-core* chỉ tính thời gian xử lý lỗi của sampler.

Theo kết quả được báo cáo trong bài báo TraStrainer, độ trễ xử lý của phương pháp này dao động trong khoảng 0.28–14.29 ms/trace [14]. Trong phép đo của khóa luận, cả *MESC-full* và *MESC-core* đều có giá trị cực đại dưới 3.3 ms/trace. Điều này có dấu hiệu tích cực khi MESC có chi phí xử lý worst-case thấp hơn đáng kể trong phép so sánh tham chiếu này.

Ngoài ra, khoảng Min–Max của TraStrainer rộng hơn so với MESC, phản ánh độ dao động thời gian lớn hơn giữa các trường hợp xử lý. Chênh lệch giữa *MESC-full* và *MESC-core* không lớn, có thể thấy chi phí tiền xử lý metric sau khi phân bổ không tạo thêm gánh nặng đáng kể trên mỗi trace. Tuy nhiên, kết quả này chỉ nên

được xem là so sánh tham chiếu, do môi trường thực nghiệm, cấu hình phần cứng và cách triển khai giữa hai hệ thống có thể khác nhau.

Tổng chi phí xử lý qua các bộ dữ liệu và mức ngân sách



Hình 4.2: Tổng chi phí xử lý của MESC trên ba bộ dữ liệu và ba mức ngân sách.

Từ Hình 4.2, tổng chi phí trên **train-ticket** duy trì ổn định ở khoảng 0.82–0.86 ms/trace. Trên hai bộ **hipster-batch1** và **hipster-batch2**, chi phí cao hơn, lần lượt khoảng 2.01–2.22 ms/trace và 2.88–3.25 ms/trace.

Xu hướng này chứng minh chi phí xử lý chịu ảnh hưởng bởi quy mô bộ dữ liệu, độ dài trace và số lượng tín hiệu cần xử lý. Tuy nhiên, khi ngân sách tăng từ 0.1% lên 2.5%, chi phí trên mỗi trace chỉ tăng nhẹ trong từng bộ dữ liệu, cho thấy phần lớn chi phí đến từ bước quét và chấm điểm trace.

4.4.5 Tổng hợp nhận xét theo câu hỏi nghiên cứu

Từ các kết quả thực nghiệm, có thể tổng hợp câu trả lời cho bốn câu hỏi nghiên cứu của khóa luận như sau.

RQ1 – So sánh hiệu suất. Kết quả thực nghiệm cho thấy hai phương pháp đề xuất có hiệu quả khác nhau tùy theo mức ngân sách và nguồn tín hiệu sử dụng. **TASD** vượt trội so với các sampler truyền thống như Random, HC, Sifter và Sieve

ở mức ngân sách thấp, chứng tỏ các tín hiệu mức trace vẫn có giá trị rõ rệt đối với RCA. Trong khi đó, **MESC** đạt kết quả nổi bật nhất ở mức ngân sách cực thấp 0.1%, với $A@1 = 41.30\%$ và $MRR = 0.5168$, tiệm cận TraStrainer đầy đủ dù không yêu cầu huấn luyện mô hình. Từ đó chỉ ra việc bổ sung metric gốc, đối sánh có dấu và đồng thuận giúp cải thiện khả năng lựa chọn trace có giá trị cho MicroRank.

RQ2 – Ảnh hưởng của ngân sách lấy mẫu. Kết quả chứng minh chất lượng RCA không tăng đơn điệu theo ngân sách. Với TASD, hiệu quả tốt nhất xuất hiện ở mức 1.0%, sau đó giảm ở mức 2.5%. Với MESC, kết quả mạnh nhất xuất hiện ở mức 0.1%, nhưng suy giảm khi ngân sách tăng lên 1.0% và 2.5%. Hiện tượng này củng cố nhận định về việc giữ thêm trace không tự động cải thiện RCA; các trace bổ sung có thể mang tín hiệu yếu, trùng lặp hoặc gây nhiễu cho quá trình đối sánh phổ lỗi. Do đó, lấy mẫu cho RCA cần kiểm soát chất lượng và cấu trúc của tập trace được giữ lại, thay vì chỉ tăng số lượng trace.

RQ3 – Đóng góp của các thành phần trong cơ chế chấm điểm. Các kết quả thực nghiệm đưa ra góc nhìn về các tín hiệu mức trace như độ trễ, cờ lỗi, độ gần của sổ sự cố và đường đi dịch vụ giúp TASD vượt nhóm sampler truyền thống trong điều kiện không có metric hệ thống. Thành phần đa dạng hóa theo đường đi dịch vụ giúp hạn chế việc chọn nhiều trace trùng lặp, trong khi các ràng buộc về trace lỗi/bình thường giúp duy trì nền đối sánh cần thiết cho MicroRank. Ở MESC, metric gốc và đối sánh có dấu đóng vai trò bổ sung bằng chứng vận hành, giúp sampler phân biệt tốt hơn các trace có giá trị RCA trong ngân sách cực thấp. Cơ chế đồng thuận tiếp tục giúp giảm dao động quyết định khi chỉ một số rất nhỏ trace được giữ lại.

RQ4 – Đánh đổi giữa chi phí và chất lượng RCA. Về chi phí lưu trữ, cả TASD và MESC đều tuân thủ ràng buộc ngân sách nghiêm ngặt, chỉ giữ lại một tỷ lệ rất nhỏ trace so với tập gốc. Về chi phí xử lý, MESC có tổng chi phí khoảng 0.8245–3.2537 ms/trace, nằm trong vùng khả thi cho thực nghiệm quy mô lớn và thấp hơn mức cực đại được báo cáo của TraStrainer [14]. Tuy nhiên, kết quả cũng cho thấy chi phí thấp và ngân sách lớn hơn không đảm bảo chất lượng RCA tốt hơn nếu tập trace sau lấy mẫu bị nhiễu hoặc mất cân bằng. Vì vậy, đánh đổi chính của bài toán không chỉ nằm giữa chi phí và số lượng trace, mà còn nằm giữa độ phủ tín hiệu, độ sạch của tập trace và khả năng duy trì nền đối sánh cho RCA.

Nhìn chung, các kết quả thực nghiệm khẳng định rằng lấy mẫu trace phục vụ RCA

không chỉ là bài toán giảm dữ liệu, mà là bài toán lựa chọn tập trace có cấu trúc phù hợp cho động cơ phân tích nguyên nhân gốc. TASD chứng minh giá trị của hướng lấy mẫu nhẹ dựa trên trace, trong khi MESC chứng minh rằng metric gốc, đối sánh có dấu và đồng thuận có thể cải thiện mạnh hiệu quả ở ngân sách cực thấp. Tuy nhiên, hiện tượng suy giảm khi ngân sách tăng cũng chỉ ra rằng các phương pháp lấy mẫu RCA-aware cần cơ chế kiểm soát phần trace bổ sung tốt hơn để duy trì độ tương phản cần thiết cho MicroRank.

Kết luận

Khóa luận đã nghiên cứu bài toán xác định dịch vụ gây lỗi trong hệ thống vi dịch vụ thông qua việc kết hợp lấy mẫu thích nghi và phân tích nguyên nhân gốc rễ. Trong hệ thống vi dịch vụ, một yêu cầu nghiệp vụ thường đi qua nhiều dịch vụ độc lập; lỗi hoặc suy giảm hiệu năng tại một dịch vụ có thể lan truyền sang các dịch vụ phụ thuộc và biểu hiện thành lỗi ở nhiều vị trí khác nhau trong chuỗi xử lý. Vì vậy, việc xác định đúng dịch vụ là nguyên nhân gốc không thể chỉ dựa trên số lỗi cục bộ, mà cần khai thác thông tin từ trace, quan hệ phụ thuộc giữa các dịch vụ, trạng thái bất thường và sự khác biệt giữa hành vi bình thường với hành vi lỗi.

Trọng tâm của khóa luận là nhìn nhận quá trình lấy mẫu *trace* không chỉ để giảm dung lượng lưu trữ, mà là bước chốt lọc dữ liệu phục vụ trực tiếp cho RCA. Bằng cách giữ lại độ tương phản giữa *trace* lỗi giúp định vị dịch vụ và *trace* bình thường làm nền đối sánh, bộ lấy mẫu bảo toàn được các thông tin cấu trúc thiết yếu. Triết lý này được hiện thực hóa qua *pipeline* hai pha: pha một lấy mẫu thích nghi theo ngân sách; pha hai đánh giá chất lượng RCA toàn trình thông qua *MicroRank* với các chỉ số A@1, A@3 và MRR. Thực nghiệm được triển khai toàn diện trên ba bộ dữ liệu *train-ticket*, *hipster-batch1*, *hipster-batch2* tại các mức ngân sách khắt khe từ 0.1% đến 2.5%.

Về mặt phương pháp, khóa luận đề xuất hai biến thể chính. Đầu tiên là **TASD**, là cấu hình nền tảng chỉ khai thác thông tin ở mức *trace*. Thông qua việc kết hợp các đặc trưng cốt lõi như độ trễ, cờ lỗi, cấu trúc *span* bằng luật kinh nghiệm và cơ chế đa dạng hóa Jaccard, TASD đảm bảo tính gọn nhẹ, dễ giải thích, không yêu cầu huấn luyện và tối ưu cho xử lý theo dòng (*streaming*). Kết quả thực nghiệm cho thấy TASD vượt trội các *sampler* truyền thống ở mức ngân sách khắt khe; cụ thể tại 0.1%, phương pháp đạt $A@1 = 23.37\%$ và $MRR = 0.4105$.

Biến thể thứ hai là **MESC**, dựa trên nền tảng TASD bổ sung thêm metric hệ thống theo dịch vụ và thời gian, cơ chế đối sánh có dấu và tổ hợp đồng thuận. Metric từ hệ thống cung cấp thêm bằng chứng về trạng thái vận hành của dịch vụ; đối sánh có dấu giúp phân biệt tín hiệu ủng hộ với tín hiệu gây nhiễu; còn cơ chế đồng thuận giúp giảm dao động quyết định ở mức ngân sách rất thấp. Ở mức 0.1%, MESC đạt $A@1 = 41.30\%$ và $MRR = 0.5168$, tiệm cận TraStrainer đầy đủ dù

không yêu cầu huấn luyện mô hình dự đoán metric. Kết quả này gợi ý rằng việc kết hợp metric gốc, đối sánh có dấu và đồng thuận có thể cải thiện đáng kể khả năng chọn trace có giá trị cho MicroRank trong điều kiện ngân sách cực nhỏ.

Các kết quả thực nghiệm cũng chỉ ra rằng lấy mẫu trace phục vụ RCA khác với phát hiện bất thường thuần túy. Nếu bộ lấy mẫu chỉ ưu tiên các trace lỗi hoặc trace bất thường nhất, tập trace sau lấy mẫu có thể bị lệch và làm MicroRank mất nền đối sánh. Chất lượng RCA phụ thuộc không chỉ vào việc giữ lại trace lỗi, mà còn phụ thuộc vào tỷ lệ trace bình thường, độ đa dạng đường đi dịch vụ, độ sạch của tập trace và sự ổn định của quyết định lấy mẫu. Một quan sát quan trọng là khi ngân sách tăng, chất lượng RCA không nhất thiết tăng theo. Cả TASD và MESC đều xuất hiện hiện tượng suy giảm ở một số mức ngân sách cao hơn, cũng có nhận định rằng việc giữ thêm trace có thể làm tăng nhiễu nếu phần trace bổ sung không mang thông tin mới hoặc làm suy yếu độ tương phản phổ lỗi.

Phân tích theo từng bộ dữ liệu cho thấy hiệu quả của phương pháp phụ thuộc vào đặc trưng workload và chất lượng tín hiệu metric. MESC cải thiện rõ rệt trên hai bộ hipster ở mức ngân sách 0.1%, nhưng không ổn định trên `train-ticket` ở các mức ngân sách cao hơn. Điều này minh chứng rằng metric gốc và đối sánh có dấu có thể mang lại lợi ích lớn khi tín hiệu metric phản ánh rõ trạng thái bất thường, nhưng cũng có thể gây nhiễu nếu ánh xạ trace-metric không đủ phù hợp với nguyên nhân gốc. Về chi phí thời gian, MESC có tổng chi phí xử lý khoảng 0.8245–3.2537 ms/trace trong các phép đo hiện có, nằm trong vùng khả thi cho thực nghiệm quy mô lớn. Khi so sánh tham chiếu với TraStrainer, chi phí của MESC thấp hơn mức cực đại được báo cáo, tuy nhiên kết quả này không nên được xem là so sánh tốc độ tuyệt đối do khác biệt về môi trường đo và cách triển khai.

Mặc dù đạt được một số kết quả tích cực, khóa luận vẫn còn một số hạn chế. Thứ nhất, quy trình đánh giá hiện tại sử dụng MicroRank làm engine duy nhất, nên kết quả có thể thay đổi nếu thay bằng một phương pháp RCA khác. Thứ hai, thực nghiệm được thực hiện trên bộ dữ liệu kiểm thử, chưa đánh giá đầy đủ trên môi trường production với tải thay đổi, thiếu metric cần thiết, metric nhiễu hoặc nhiễu loại lỗi phức tạp hơn. Thứ ba, biểu diễn trace chủ yếu ở mức tổng hợp, chưa khai thác sâu cấu trúc span, quan hệ cha-con và các kiểu đồ thị trong trace. Thứ tư, một số trọng số, ngưỡng và ràng buộc vẫn được thiết kế theo luật kinh nghiệm, do đó cần được kiểm chứng thêm khi áp dụng trên các hệ thống khác.

Trong tương lai, phương pháp có thể được phát triển theo một số hướng. Trước hết, có thể cải thiện biểu diễn trace bằng các đặc trưng giàu ngữ nghĩa hơn, chẳng hạn *trace embedding*, *graph embedding* hoặc biểu diễn dựa trên cấu trúc span. Tiếp theo, cần nghiên cứu cơ chế mở rộng ngân sách có kiểm soát, nhằm bảo đảm rằng phần trace bổ sung khi tăng ngân sách thực sự làm tăng độ phủ tín hiệu thay vì làm tăng nhiễu. Ngoài ra, cần mở rộng đánh giá sang nhiều engine RCA, nhiều bộ dữ liệu thực tế và nhiều loại sự cố hơn để kiểm tra khả năng tổng quát hóa. Cuối cùng, phương pháp có thể được tích hợp sâu hơn vào các quy trình quan sát như OpenTelemetry Collector hoặc SigNoz để tiến gần hơn tới triển khai thực tế.

Tóm lại, khóa luận đã đề xuất một hướng tiếp cận lấy mẫu trace nhận biết RCA, trong đó bộ lấy mẫu không chỉ nhằm giảm chi phí lưu trữ mà còn chủ động bảo toàn thông tin cần thiết cho phân tích nguyên nhân gốc. TASD cung cấp một nền tảng lấy mẫu nhẹ, dễ giải thích và khả thi trong điều kiện chỉ có dữ liệu trace. MESC mở rộng nền tảng đó bằng metric gốc, đối sánh có dấu và đồng thuận tổ hợp để cải thiện chất lượng lựa chọn trace ở ngân sách rất thấp. Kết quả thực nghiệm cho thấy thiết kế bộ lấy mẫu theo mục tiêu phục vụ cho việc phân tích RCA ở phần sau là một hướng tiếp cận có ý nghĩa, đồng thời chỉ ra rằng chất lượng RCA phụ thuộc mạnh vào cấu trúc và giá trị tín hiệu của tập trace sau lấy mẫu, không chỉ phụ thuộc vào số lượng trace được giữ lại.

Tài liệu tham khảo

- [1] OpenObserve. Head-based and tail-based sampling in distributed tracing. <https://openobserve.ai/blog/head-and-tail-based-sampling/>, 2026.
- [2] Jaeger. Terminology: Traces and spans. <https://www.jaegertracing.io/docs/2.dev/architecture/terminology/>, 2026. Accessed: 2026-04-30.
- [3] Guangba Yu, Pengfei Chen, Hongyang Chen, Zijie Guan, Zicheng Huang, Linxiao Jing, Tianjun Weng, Xinmeng Sun, and Xiaoyun Li. Microrank: End-to-end latency issue localization with extended spectrum analysis in microservice environments. In *Proceedings of The Web Conference 2021*, pages 3087–3098. ACM, 2021.
- [4] James Lewis and Martin Fowler. Microservices: A definition of this new architectural term. <https://martinfowler.com/articles/microservices.html>, 2014.
- [5] Nicola Dragoni, Saverio Giallorenzo, Alberto Lluch Lafuente, Manuel Mazzara, Fabrizio Montesi, Ruslan Mustafin, and Larisa Safina. Microservices: Yesterday, today, and tomorrow. In *Present and Ulterior Software Engineering*, pages 195–216. Springer, 2017.
- [6] Benjamin H. Sigelman, Luiz André Barroso, Mike Burrows, Pat Stephenson, Manoj Plakal, Donald Beaver, Saul Jaspan, and Chandan Shanbhag. Dapper, a large-scale distributed systems tracing infrastructure. Technical Report dapper-2010-1, Google, 2010.
- [7] Zeyan Li, Junjie Chen, Rui Jiao, Nengwen Zhao, Zhijun Wang, Shuwei Zhang, Yanjun Wu, Long Jiang, Leiqin Yan, Zikai Wang, Zhe Tian, Rongjia Zhang, Li Zhang, Pinjia He, Jiazhen Zhao, Mingjie Li, and Yingnong Dang. Practical root cause localization for microservice systems via trace analysis. In *Proceedings of the 2021 IEEE/ACM International Symposium on Quality of Service (IWQOS)*. IEEE, 2021.
- [8] Ping Liu, Haowen Xu, Qianyu Ouyang, Rui Jiao, Zhekang Chen, Shenglin Zhang, Jiahai Yang, Linlin Mo, Jice Zeng, Wenman Xue, and Dan Pei. Unsupervised detection of microservice trace anomalies through service-level deep

- bayesian networks. In *2020 IEEE 31st International Symposium on Software Reliability Engineering*. IEEE, 2020.
- [9] OpenTelemetry. Sampling. <https://opentelemetry.io/docs/concepts/sampling/>, 2025.
- [10] Jaeger. Sampling. <https://www.jaegertracing.io/docs/2.5/architecture/sampling/>, 2025.
- [11] Yu Gan, Mingyu Liang, Sundar Dev, David Lo, and Christina Delimitrou. Sage: Practical and scalable ml-driven performance debugging in microservices. In *Proceedings of the 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, pages 135–151. ACM, 2021.
- [12] Raja R. Sambasivan, Rodrigo Fonseca, Ilari Shafer, and Gregory R. Ganger. So, you want to trace your distributed system? key design insights from years of practical experience. Technical Report CMU-PDL-14-102, Carnegie Mellon University, 2014.
- [13] Pedro Las-Casas, Giorgi Papakerashvili, Vaastav Anand, and Jonathan Mace. Sifter: Scalable sampling for distributed traces, without feature engineering. In *Proceedings of the ACM Symposium on Cloud Computing*, pages 312–324. ACM, 2019.
- [14] Haiyu Huang, Xiaoyu Zhang, Pengfei Chen, Zilong He, Zhiming Chen, Guangba Yu, Hongyang Chen, and Chen Sun. Trastrainer: Adaptive sampling for distributed traces with system runtime state. *Proceedings of the ACM on Software Engineering*, 1(FSE):473–493, 2024.
- [15] OpenTelemetry. Tail sampling with opentelemetry: Why it’s useful, how to do it, and what to consider. <https://opentelemetry.io/blog/2022/tail-sampling/>, 2022.
- [16] Zicheng Huang, Pengfei Chen, Guangba Yu, Hongyang Chen, and Zibin Zheng. Sieve: Attention-based sampling of end-to-end trace data in distributed microservice systems. In *Proceedings of the 2021 IEEE International Conference on Web Services (ICWS)*, pages 436–446. IEEE, 2021.

- [17] Alim Ul Gias, Yicheng Gao, Matthew Sheldon, José A. Perusquía, Owen O’Brien, and Giuliano Casale. Samplehst: Efficient on-the-fly selection of distributed traces. In *NOMS 2023-2023 IEEE/IFIP Network Operations and Management Symposium*, pages 1–9. IEEE, 2023.
- [18] Zhuangbin Chen, Zhihan Jiang, Yuxin Su, Michael R. Lyu, and Zibin Zheng. Tracemesh: Scalable and streaming sampling for distributed traces. In *Proceedings of the 2024 IEEE 17th International Conference on Cloud Computing (CLOUD)*, pages 54–65. IEEE, 2024.
- [19] Xiang Zhou, Xin Peng, Tao Xie, Jun Sun, Chao Ji, Wenhai Li, and Dan Ding. Benchmarking microservice systems for software engineering research. In *Proceedings of the 40th International Conference on Software Engineering: Companion Proceedings*, pages 323–324. ACM, 2018.
- [20] Ping Wang, Jingmin Xu, Meng Ma, Weilan Lin, Disheng Pan, Yuan Wang, and Pengfei Chen. Cloudranger: Root cause identification for cloud native systems. In *2018 18th IEEE/ACM International Symposium on Cluster, Cloud and Grid Computing (CCGRID)*, pages 492–502. IEEE, 2018.
- [21] OpenTelemetry. Opentelemetry concepts. <https://opentelemetry.io/docs/concepts/>, 2026.
- [22] James A. Jones, Mary Jean Harrold, and John Stasko. Visualization of test information to assist fault localization. In *Proceedings of the 24th International Conference on Software Engineering*, pages 467–477. ACM, 2002.
- [23] Shuaiyu Xie, Jian Wang, Maodong Li, Peiran Chen, Jifeng Xuan, and Bing Li. Tracepicker: Optimization-based trace sampling for microservice-based systems. *Proceedings of the ACM on Software Engineering*, 2(FSE):1802–1823, 2025.
- [24] IBM. What is jaccard similarity? <https://www.ibm.com/think/topics/jaccard-similarity>, 2024.
- [25] Jure Leskovec, Anand Rajaraman, and Jeffrey D. Ullman. *Mining of Massive Datasets*. Cambridge University Press, 3 edition, 2020.
- [26] Thomas G. Dietterich. Ensemble methods in machine learning. *Multiple Classifier Systems*, pages 1–15, 2000.

- [27] W. Eric Wong, Vidroha Debroy, Ruizhi Gao, and Yihao Li. The dstar method for effective software fault localization. *IEEE Transactions on Reliability*, 63(1):290–308, 2014.
- [28] Lawrence Page, Sergey Brin, Rajeev Motwani, and Terry Winograd. The pagerank citation ranking: Bringing order to the web. *Stanford InfoLab*, 1999.
- [29] OpenTelemetry. Opentelemetry collector architecture. <https://opentelemetry.io/docs/collector/architecture/>, 2026. Truy cập ngày 07/05/2026.
- [30] Google Cloud Platform. Online boutique: Cloud-first microservices demo application. <https://github.com/GoogleCloudPlatform/microservices-demo>, 2026.
- [31] IntelligentDDS. Trastrainer: The open-source version of trastrainer. <https://github.com/IntelligentDDS/TraStrainer>, 2024.